

Université Marie et Louis Pasteur

UFR Sciences et Techniques
Laboratoire de Mathématiques Besançon

Spécialité : Modélisation statistique

Rapport Stage de Master 2

Approximation statistique de champ de vecteurs de courants océaniques

Étudiant	Kuassi Pierre DOVODJI Courriel : kuassi_pierre.dovodji@edu.univ-fcomte.fr
Entreprise d'accueil	Institut de Mathématique de Bourgogne (IMB) Mediterranean Institute of Oceanography (MIO)
Maîtres de stage	Hervé Cardot (IMB), Professeur des universités Courriel : Herve.Cardot@u-bourgogne.fr David Nerini (MIO), Maître de conférences Courriel : david.nerini@univ-amu.fr
Référente à l'université	Camélia Goga , Professeure des universités Courriel : camelia.goga@univ-fcomte.fr
Date et heure de la soutenance	28/08/2025 à 14h

Remerciements

Je tiens tout d'abord à exprimer ma profonde gratitude à toute l'équipe pédagogique du Master Modélisation Statistique de Besançon. Et plus particulièrement à la professeure Camélia Goga, pour ses conseils avisés, son écoute attentive et son encouragement constant tout au long de ce parcours.

Avant d'entamer ce rapport, je souhaite également remercier chaleureusement mes tuteurs de stage, le professeur Hervé Cardot et le docteur David Nerini, pour leur générosité en matière de formation et d'encadrement. Leur soutien, leurs conseils éclairés et la confiance qu'ils m'ont accordée ont été essentiels dans la réalisation des missions présentées dans ce mémoire.

Enfin, je tiens à remercier tous mes professeurs, qui ont su m'inciter à travailler avec rigueur, en partageant généreusement leur expérience et leurs compétences, ce qui a largement contribué à enrichir mon apprentissage.

Résumé

Ce mémoire porte sur l'estimation et l'approximation de champs vectoriels à partir de données partielles et bruitées. Plusieurs méthodes d'interpolation et d'approximation sont étudiées, notamment les splines plaque mince et les méthodes intégrant des contraintes physiques via la norme div-curl. L'analyse met en évidence la sensibilité des méthodes classiques au bruit et à la répartition des données, soulignant l'importance d'introduire des régularisations physiques pour améliorer la robustesse des estimations. Les résultats montrent que la méthode proposée, prenant en compte les propriétés physiques du champ, offre une meilleure précision et stabilité. Des perspectives sont proposées pour l'intégration de données indirectes et l'optimisation des schémas d'échantillonnage.

Abstract

This thesis focuses on the estimation and approximation of vector fields from partial and noisy data. Several interpolation and approximation methods are studied, including thin-plate splines and approaches incorporating physical constraints via the div-curl norm. The analysis highlights the sensitivity of classical methods to noise and data distribution, emphasizing the need to introduce physical regularizations to improve estimation robustness. Results show that the proposed method, which takes into account the physical properties of the field, provides better accuracy and stability. Perspectives include integrating indirect data and optimizing sampling schemes.

Mots-clés : Estimation de champ vectoriel, Interpolation, Approximation, Splines plaque mince, Contraintes physiques, Norme div-curl, Bruit, Échantillonnage, Systématique, Aléatoire

Keywords : Vector field estimation, Interpolation, Approximation, Thin-plate splines, Physical constraints, Div-curl norm, Noise, Sampling, Systematic, Random

Table des matières

Introduction	9
1 Cadre général du stage	11
1.1 Présentation des laboratoires de recherche	11
1.2 Sujet de stage	12
1.3 Outils mathématiques	14
1.3.1 Champ de vecteurs	14
1.3.2 Divergence	14
1.3.3 Rotationnel	15
1.3.4 La norme div-curl	15
1.3.5 Fonctions splines	16
1.3.6 Fonction de base radiale	17
1.3.7 Définitions des espaces fonctionnels[4]	18
1.3.8 Loi de von Mises	19
2 Présentation des données et méthodologies	20
2.1 Jeux de données	20
2.1.1 Observation directe	21
2.1.2 Observations indirectes	22
2.2 Méthode d'interpolation	23
2.2.1 Interpolation bilinéaire	24
2.2.2 Interpolation avec B-splines	26
2.2.3 Méthode de L. Amodéi 1991 [1]	27
2.3 Méthodes d'approximation	30
2.3.1 Splines plaque mince	31
2.3.2 Algorithme d'estimation du champ vectoriel	34
2.4 Méthode tenant compte des propriétés physiques	34
3 Application	37
3.1 Métriques	37
3.1.1 Erreur Quadratique Moyenne (EQM)	37
3.1.2 Erreur Angulaire Moyenne(EAM)	37
3.2 Protocole expérimental et évaluation	38
3.3 Résultats	39
3.3.1 Analyse des résultats de l'application des méthodes sur les données	39

3.3.2	Analyse de la sensibilité au bruit	40
3.4	Discussion	43
3.5	Difficultés rencontrées	43
3.6	Suggestions	43
Conclusion		45
Bibliographie		47
Annexe		48

Table des figures

Figure 1.1 :	L'Institut de Mathématiques de Bourgogne	12
Figure 1.2 :	Activité de terrain au MIO – Plongée pour l'observation et l'échantillonnage.	13
Figure 1.3 :	Exemple de champ partiel et champ reconstitué.	13
Figure 2.1 :	Champ de courant récupéré sur copernicus.	21
Figure 2.2 :	Échantillonnage systématique.	22
Figure 2.3 :	Échantillonnage aléatoire.	22
Figure 2.4 :	Extrait des séquences de test : à gauche, OPA ; à droite, METEOSAT. [10]	23
Figure 2.5 :	Échantillonnage systématique-Interpolation bilinéaire.	25
Figure 2.6 :	Sous-échantillonnage aléatoire - Méthode des barycentres.	26
Figure 2.7 :	Échantillonnage systématique - Interpolation avec B-spline.	27
Figure 2.8 :	Échantillonnage aléatoire - Interpolation avec B-spline.	27
Figure 2.9 :	Échantillonnage systématique - Méthode d'Amodéi.	29
Figure 2.10 :	Échantillonnage aléatoire - Méthode d'Amodéi.	29
Figure 2.11 :	Échantillonnage systématique - Interpolation spline plaque mince.	32
Figure 2.12 :	Échantillonnage aléatoire - Interpolation spline plaque mince.	33
Figure 2.13 :	Échantillonnage systématique - Approximation spline plaque mince.	33
Figure 2.14 :	Échantillonnage aléatoire - Approximation spline plaque mince.	33
Figure 3.1 :	Bruitage avec la loi von Mises.	38
Figure 3.2 :	Comparaison des erreurs sous variations de bruit pour échantillonnage aléatoire et systématique	41
Figure 3.3 :	Comparaison AEM/EQM pour échantillonnages systématique et aléatoire.	42
Figure 3.4 :	Méthode d'Amodéi — Échantillonnage aléatoire	49
Figure 3.5 :	Méthode d'Amodéi — Échantillonnage systématique	49
Figure 3.6 :	Interpolation bilinéaire — Échantillonnage aléatoire	50
Figure 3.7 :	Interpolation bilinéaire — Échantillonnage systématique	50
Figure 3.8 :	Interpolation B-spline	50

Liste des tableaux

Tableau 3.1 :	EAM - Échantillonnage systématique	39
Tableau 3.2 :	EAM - Échantillonnage aléatoire	39
Tableau 3.3 :	EQM - Échantillonnage Systématique	39
Tableau 3.4 :	EQM - Échantillonnage aléatoire	39
Tableau 3.5 :	EAM - Échantillonnage systématique + bruit	40
Tableau 3.6 :	EAM - Échantillonnage aléatoire + bruit	40
Tableau 3.7 :	EQM - Échantillonnage Systématique + bruit	40
Tableau 3.8 :	EQM - Échantillonnage aléatoire + bruit	40

Notation et conventions

$\mathbf{x}$: désigne un vecteur de $\mathbb{R}^2$, c'est-à-dire $\mathbf{x} = (x, y)$.

$\mathbf{w}$: $\mathbf{w} = \mathbf{w}(\mathbf{x}) = (u(\mathbf{x}), v(\mathbf{x}))$

$\phi_x = \frac{\partial \phi}{\partial x}$, $\phi_y = \frac{\partial \phi}{\partial y}$, $\phi_{xx} = \frac{\partial^2 \phi}{\partial x^2}$, $\phi_{yy} = \frac{\partial^2 \phi}{\partial y^2}$, $\phi_{xy} = \frac{\partial^2 \phi}{\partial x \partial y}$

$\|\cdot\|$: la norme euclidienne.

INTRODUCTION

Les océans, qui recouvrent plus de 70 % de la surface de la Terre, constituent un pilier essentiel du système climatique planétaire et abritent une biodiversité variée, y compris notamment des espèces comme les éléphants de mer. Sa dynamique combinée à celui de l'atmosphère, intimement liée aux interactions entre courants marins et les vents, joue un rôle central dans la régulation du climat, la navigation et la gestion des ressources marines ce qui fait de la connaissance de la circulation des océans un enjeu important[11]. Aujourd'hui, plusieurs programmes et structures de recherche en France s'intéressent aux enjeux environnementaux et à l'étude des océans. Parmi eux, l'Institut des Mathématiques pour la Planète Terre (iMPT), créé en 2021 par le Centre National de la Recherche Scientifique (CNRS) et plusieurs universités, fédère la communauté scientifique autour de l'application des mathématiques à la compréhension et à la préservation de l'environnement. Ses travaux couvrent des thématiques allant de la modélisation des dynamiques océaniques et atmosphériques, à l'étude de la biodiversité et des risques liés au changement climatique. Parmi les nombreux sujets étudiés en océanographie et en sciences de l'atmosphère, les champs de courants sont particulièrement importants. Ils montrent comment l'eau dans les océans ou l'air dans l'atmosphère se déplace, en indiquant leur vitesse et leur direction. Comprendre ces mouvements est essentiel pour mieux connaître le fonctionnement des océans, du vent, et comment ils interagissent entre eux.

Aussi, afin d'étudier les comportements alimentaires des éléphants de mer, des capteurs qui permettent d'enregistrer leurs déplacements dans l'océan Austral ont été placés sur eux depuis une dizaine d'années. Ces mammifères marins parcourent de longues distances, parfois plusieurs dizaines de milliers de kilomètres, en quête de nourriture, tout en plongeant régulièrement sur de longues durées[8]. En croisant ces trajectoires avec des données océanographiques, il est possible d'interpréter certains comportements, notamment ceux associés à la présence de structures océaniques telles que les gyres. Pour ce faire, on utilise des champs de courants issus de données satellitaires correspondant aux instants de passage des animaux. Toutefois, ces données sont souvent incomplètes, en raison de la couverture nuageuse fréquente dans ces régions, ce qui limite leur utilité.

Ce stage vise à reconstituer des champs vectoriels de vitesse précis à partir de ces données incomplètes. Pour cela, nous proposons des méthodes d'approximation par splines vectorielles, permettant d'interpoler les champs tout en intégrant des contraintes physiques par exemple sur la divergence ou le rotationnel, afin d'assurer leur cohérence avec les lois de la physique [10].

Les méthodes seront testées sur des champs de vitesses océaniques issus de modèles numériques, disponibles

sur la plate-forme Copernicus¹

Ce mémoire est organisé en trois chapitres, outre cette introduction. Le premier chapitre présente le sujet de stage, les concepts théoriques et les outils mathématiques essentiels à la réalisation de notre travail. Ensuite, le deuxième chapitre présente les données utilisées et les différentes méthodologies d'interpolation ou d'approximation mises en œuvre. Enfin, dans le troisième chapitre, une étude complète des résultats obtenus avec les différentes méthodes est réalisée, ainsi qu'une analyse plus fine de la performance de chacune d'elles vis-à-vis du bruit ajouté et du degré d'observation partielle, de même qu'une présentation des limites et des perspectives d'amélioration des techniques proposées. Enfin, une conclusion générale et les références bibliographiques sont fournies.

1. Copernicus est un programme européen d'observation de la Terre qui fournit des données environnementales issues de satellites, de capteurs au sol et de modèles numériques. Il offre un accès libre et gratuit à des informations précises sur l'océan, l'atmosphère, les terres émergées et le changement climatique.

CHAPITRE 1

CADRE GÉNÉRAL DU STAGE

Ce chapitre est consacré à la présentation du cadre général du déroulement de ce stage. Il s'agit dans un premier temps, de décrire les laboratoires au sein desquels le stage s'effectue, ensuite, présenter le sujet de stage et enfin définir les concepts théoriques et les outils mathématiques nécessaires à la compréhension et à la mise en œuvre de notre travail.

1.1 Présentation des laboratoires de recherche

Ce stage a été réalisé au sein de l'Institut de Mathématiques de Bourgogne (IMB), situé à l'Université Bourgogne Europe, Dijon. Il s'inscrit dans le cadre d'un projet de recherche porté par l'Institut Méditerranéen d'Océanographie (MIO), Marseille et l'Institut de mathématiques de Bourgogne et financé par le Centre National de la Recherche Scientifique (CNRS) par le biais de l'institut des Mathématiques pour la Planète Terre (iMPT). Ce projet est coordonné par David Nérini (MIO) et Hervé Cardot (IMB) qui encadrent ce stage. Il porte sur les développements méthodologiques pour l'analyse de données en océanographie le cas de données échantillonnées en Antarctique par des mammifères marins.

L'IMB est un laboratoire de recherche sous la tutelle du CNRS et de l'Université Bourgogne Europe, créé en 2003 qui à la date d'aujourd'hui est composé de 51 enseignants-chercheurs, 4 chercheurs CNRS, 25 doctorants et 6 post-doctorants. Il est constitué de trois équipes de recherche qui sont :

- Géométrie, Algèbre, Dynamique et Topologie (GADT) qui s'occupe des questions liées à la géométrie, les équations différentielles, la théorie des nombres et la topologie.
- Mathématique Physique (MP) qui couvre les thématiques de la physique mathématique et certains problèmes mathématiques issus d'autres disciplines scientifiques.
- Statistique, Probabilités, Optimisation et Contrôle (SPOC) qui se concentre essentiellement sur les thématiques liées à la statistique, des probabilités, de l'optimisation, du contrôle optimal, du calcul scientifique et de l'analyse numérique.

Le MIO est un laboratoire de recherche en Océanologie sous la tutelle des Universités d'Aix-Marseille, de Toulon, du CNRS et de l'IRD. Son but est d'appréhender l'environnement marin ainsi que son évolution en réponse au changement global. Il est composé d'un effectif de 250 enseignants chercheurs des universités, près de cinquante

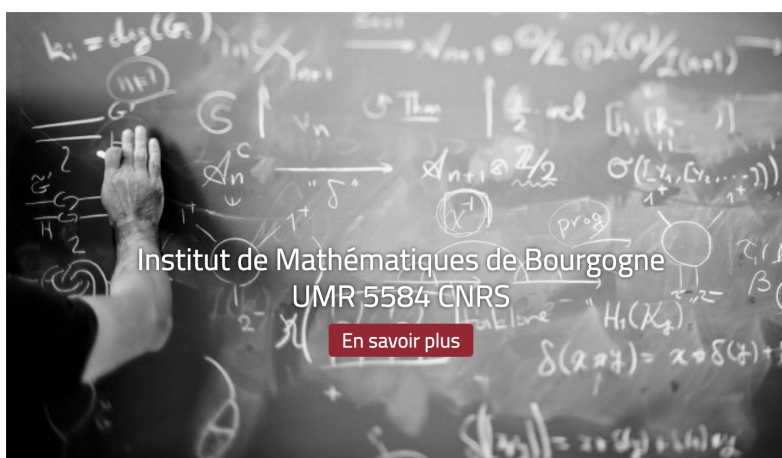


Figure 1.1 – L’Institut de Mathématiques de Bourgogne

ingénieurs et techniciens, plus de soixante-dix doctorants et post-doctorants, ainsi qu’une équipe d’administrateurs au service des activités de recherche et est constitué de six équipes de recherches qui sont :

- Océanographie Physique, Littorale et Côtière (OPLC) qui étudie les processus multi-échelles et leurs interactions dans la dynamique de l’océan et avec l’atmosphère en combinant les observations, la modélisation numérique, l’intelligence artificielle pour comprendre les échanges d’énergie et leurs effets climatiques et sociétaux.
- Chimie des Environnements Marins (CEM) étudie la composition organique et inorganique du milieu marin.
- Microbiologie Environnementale Bioprocédés (MEB) explore la diversité microbienne taxonomique et fonctionnelle dans les océans, avec une expertise historique dans l’isolement et la caractérisation indispensable de modèles cultivés.
- BIOGEOCHIMIE s’intéresse aux processus biotiques et abiotiques qui contrôlent les cycles biogéochimiques des éléments dans l’océan et sur le couplage entre ces cycles à différentes échelles spatio-temporelles
- Écologie Marine et Biodiversité (EMBIO) qui a pour objectif de comprendre le rôle de la composante biotique dans le fonctionnement des écosystèmes marins, dans le contexte du changement global.
- ECOlogie Théorique, Modélisation & Analyse de Données (ECOMAD) développe des méthodes interdisciplinaires qui associent modélisation, analyse de données et écologie théorique pour comprendre comment les écosystèmes marins réagissent aux perturbations, tout en valorisant les données historiques, les nouveaux capteurs et en consolidant l’expertise du MIO à l’échelle nationale et internationale.

1.2 Sujet de stage

En dehors des approches expérimentales en recherche scientifique, la dynamique des courants océaniques est capitale pour l’ensemble des activités humaines. En effet, à l’échelle du système climatique, leur rôle est déterminant pour la redistribution de chaleur, d’éléments nutritifs et d’espèces marines ; dans le domaine du transport maritime, savoir utiliser les courants de surface permet d’optimiser les routes de navigation pour diminuer le temps de trajet, réduire la consommation en carburant et éviter certaines conditions extrêmes. Par ailleurs, les connaissances les plus fines sur le comportement des mammifères marins dans l’océan Austral reposent sur des champs de vitesse océanique de référence. La connaissance des courants occupe donc une place stratégique tant pour



Figure 1.2 – Activité de terrain au MIO – Plongée pour l'observation et l'échantillonnage.

garantir la sécurité que pour accroître la compétitivité économique.

Il existe aujourd'hui des modèles physiques complexes, issus de la dynamique des fluides géophysiques, qui permettent de générer des champs de courant à grande échelle qui sont principalement basés sur le principe d'assimilation de données¹. Toutefois, ces modèles reposent sur des hypothèses, des forçages atmosphériques de surface, les conditions aux limites et des paramétrisations de la physique non résolue ; qui peuvent s'éloigner des échelles fines nécessaires à l'analyse comportementale ou écologique. De plus, ils peuvent manquer de précision localement, ou ne pas refléter fidèlement les observations ponctuelles disponibles.

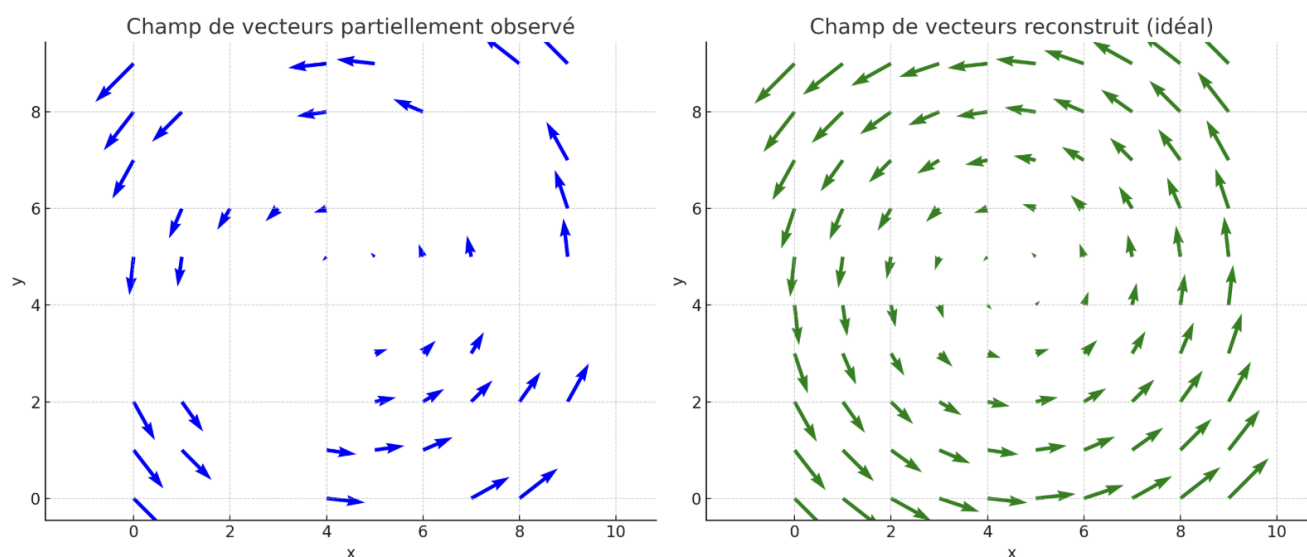


Figure 1.3 – Exemple de champ partiel et champ reconstitué.

1. Méthode qui consiste à combiner des observations réelles avec des prévisions issues d'un modèle afin d'obtenir une estimation plus précise de l'état d'un système dynamique.

Dans ce contexte, une question centrale se pose : **comment reconstituer, à partir de données incomplètes ou bruitées, des champs vectoriels de courant marin cohérents, à la fois fidèles aux observations et compatibles avec les contraintes physiques de l'écoulement ?**

Cette problématique soulève plusieurs défis :

- Comment intégrer des observations incomplètes tout en assurant la cohérence spatiale du champ reconstitué ?
- Comment incorporer des contraintes physiques pour guider la reconstruction ?
- Et comment évaluer la qualité des champs interpolés en l'absence de vérité terrain direct ?

L'objectif de ce travail est de proposer des méthodes mathématiques d'estimation, basées sur des outils souples visant à reconstituer des champs cohérents et adaptés aux données disponibles, tout en intégrant des principes physiques fondamentaux. Elles offrent ainsi une alternative ou un complément aux modèles physiques classiques, plus directement exploitables pour l'analyse des comportements marins, mais aussi pour des applications pratiques en océanographie.

1.3 Outils mathématiques

Nous commencerons par poser les bases en présentant les notions fondamentales.

1.3.1 Champ de vecteurs

Définition 1.3.1. On appelle *champ* une application $\mathbf{w}$ qui associe à chaque point $\mathbf{x} \in \Omega \subset \mathbb{R}^d$, une grandeur scalaire ou vectorielle.

$$\mathbf{w} : \Omega \rightarrow \mathbb{R}^{d'}$$

Ω est le domaine sur lequel le champ est défini, $d, d' \in \mathbb{N}^*$ et $d \geq 1$

- **Champ scalaire** : une seule valeur numérique à chaque point (par exemple, température, pression). Autrement dit $d' = 1$
- **Champ vectoriel** : un vecteur à chaque point c'est-à-dire $d' > 1$ (par exemple, la vitesse d'un fluide).

Dans ce travail, nous nous concentrons uniquement sur l'étude de champs vectoriels définis de $\Omega \subset \mathbb{R}^2$ dans $\mathbb{R}^2$, c'est-à-dire des champs qui associent à chaque point du plan un vecteur du plan.

1.3.2 Divergence

Définition 1.3.2. L'opérateur *divergence* est un opérateur différentiel qui, à un champ vectoriel $\mathbf{w}$ de $\mathbb{R}^d$ dans $\mathbb{R}^d$ ($d \in \mathbb{N}^*$), associe un champ scalaire noté $\text{div } \mathbf{w}$.

Si

$$\mathbf{w}(\mathbf{x}) = (w_1(\mathbf{x}), w_2(\mathbf{x}), \dots, w_d(\mathbf{x})), \quad \mathbf{x} = (x_1, x_2, \dots, x_d).$$

Alors

$$\text{div } \mathbf{w} = \frac{\partial w_1}{\partial x_1} + \frac{\partial w_2}{\partial x_2} + \dots + \frac{\partial w_d}{\partial x_d}.$$

Cette quantité mesure la tendance d'un point du champ à "émettre" ou "absorber" du flux. Sa signification physique est la suivante :

- Si $\text{div } \mathbf{w} > 0$ en un point, le vecteur semble s'éloigner de ce point, créant un effet centrifuge, comme si le point était une source.
- Si $\text{div } \mathbf{w} < 0$, le vecteur semble converger vers ce point, générant un effet centripète, formant un puits.
- Si $\text{div } \mathbf{w} = 0$, il n'y a ni source ni puits localement, le flux est conservatif.

1.3.3 Rotationnel

Définition 1.3.3. *L'opérateur rotationnel est un opérateur différentiel aux dérivées partielles qui, à un champ vectoriel $\mathbf{w}$ de $\mathbb{R}^d$ dans $\mathbb{R}^d$ ($d = 2$ ou 3), fait correspondre un autre champ noté $\text{rot } \mathbf{w}$.*

▷ **En dimension 2** Pour $\mathbf{w}(\mathbf{x}) = (w_1, w_2)$, on a

$$\text{rot } \mathbf{w} = \frac{\partial w_1}{\partial x_1} - \frac{\partial w_2}{\partial x_2}$$

▷ **En dimension 3** Pour $\mathbf{w}(\mathbf{x}) = (w_1, w_2, w_3)$, on a :

$$\text{rot } \mathbf{w} = \left(\frac{\partial w_3}{\partial x_2} - \frac{\partial w_2}{\partial x_3}, \frac{\partial w_1}{\partial x_3} - \frac{\partial w_3}{\partial x_1}, \frac{\partial w_2}{\partial x_1} - \frac{\partial w_1}{\partial x_2} \right)$$

Le **rotationnel d'un champ vectoriel** mesure la tendance des vecteurs à tourner autour d'un point. Si $\mathbf{w}$ représente la vitesse d'un fluide :

- $\text{curl } \mathbf{w}$ est un vecteur dont la direction indique l'axe de rotation locale.
- Sa norme indique l'intensité du tourbillon autour de ce point.
- Si $\text{curl } \mathbf{w} \neq 0$: les particules de fluide subissent une rotation locale, formant un tourbillon.
- Si $\text{curl } \mathbf{w} = 0$: le fluide est irrotationnel, il peut se déplacer, mais sans créer de petits tourbillons locaux.

Pour plus de détails sur ces opérateurs, on se référera à [2], qui a servi de source pour la rédaction.

1.3.4 La norme div-curl

Définition 1.3.4. *Soit $\Omega \subset \mathbb{R}^2$ un domaine ouvert borné régulier, et soit $\mathbf{w} = (u, v) \in H^2(\Omega)^2$. On appelle semi-norme div-curl de $\mathbf{w}$ la quantité suivante :*

$$\|\mathbf{w}\|_d := \alpha \|\nabla \text{div } \mathbf{w}\|^2 + \beta \|\nabla \text{rot } \mathbf{w}\|^2,$$

avec $\alpha, \beta \geq 0$

Cette norme mesure la régularité physique d'un champ vectoriel en termes de divergence et de rotation. Elle sert à contrôler la variation spatiale des sources/puits et des tourbillons. Dans le contexte des approximations numériques comme dans notre cas, cette norme permet de guider la construction des champs vectoriels simulés pour qu'ils restent cohérents avec la physique du problème, elle pénalise les variations brusques de divergence ou de rotation, garantissant que les champs approchés respectent à la fois les propriétés d'incompressibilité, de rotation locale souhaitées.

1.3.5 Fonctions splines

Depuis les débuts de la théorie de l'approximation, les bases polynomiales occupent une place importante. Théoriquement, les polynômes permettent d'approcher toute fonction continue définie sur un intervalle fermé $[a, b]$, conformément au théorème de Weierstrass voir Schumaker[9], qui garantit qu'une telle approximation peut atteindre n'importe quel degré de précision.

Cependant, malgré leurs nombreuses qualités : simplicité de manipulation, continuité, différentiabilité les polynômes souffrent d'un manque de robustesse lorsqu'ils sont utilisés pour approcher des fonctions sur de larges intervalles. En particulier, ils peuvent présenter des oscillations importantes (phénomène dit de Runge) dès que l'on augmente leur degré.

Pour remédier à ce défaut, une méthode efficace consiste à découper le domaine en plusieurs sous-intervalles de sorte que l'on utilise des polynômes de faible degré sur chacun de ces morceaux qui permettent d'approcher localement la fonction d'intérêt. Ceci amène naturellement à la définition des splines, introduite par Schoenberg (1946a, b). Ce sont des fonctions définies par morceaux et constituées de polynômes sur les sous-intervalles, adaptées localement, mais assemblées de façon à obtenir la continuité et la dérivabilité sur tout le domaine. Ainsi, les splines sont capables de combiner la souplesse locale des polynômes de faible degré et une bonne stabilité numérique.

Définition 1.3.5. [3] Soit une série de points de données $(x_0, y_0), (x_1, y_1), \dots, (x_n, y_n)$, avec $a = x_0 < x_1 < \dots < x_n = b$.

Une spline d'interpolation d'ordre m est une fonction $S(x)$ définie par morceaux, composée de polynômes de degré m sur chaque intervalle $[x_i, x_{i+1}]$, telle que :

- $S(x_i) = y_i$ pour tous les $i = 0, \dots, n$ (interpolation),
- $S(x)$ est de classe C^{m-1} sur $[x_0, x_n]$ (continuité des dérivées jusqu'à l'ordre $m - 1$).

Définition 1.3.6. Étant donné des noeuds $0 \leq t_0 \leq t_1 \leq \dots \leq t_n \leq 1$.

Les B-splines normalisées d'ordre m sont définies récursivement par la relation suivante :

$$B_i^{(m)}(x) = \frac{x - t_i}{t_{i+m-1} - t_i} B_i^{(m-1)}(x) + \frac{t_{i+m} - x}{t_{i+m} - t_{i+1}} B_{i+1}^{(m-1)}(x),$$

avec la condition initiale :

$$B_i^{(1)}(x) = \begin{cases} 1, & \text{si } t_i \leq x < t_{i+1}, \\ 0, & \text{sinon.} \end{cases}$$

On parle de B-spline normalisée car

$$\sum_i B_i^m(t) = 1 \quad \text{pour tout } t$$

Par convention, lorsque les noeuds sont confondus (multiplicité des noeuds), nous posons $\frac{0}{0} = 0$

Remarque 1.3.1. D'après [6], toute fonction spline s d'ordre m , définie sur un intervalle borné $[a, b]$ avec un vecteur de noeuds

$$\xi = (\xi_0, \xi_1, \dots, \xi_k),$$

peut s'exprimer comme une combinaison linéaire de fonctions B-splines :

$$s(x) = \sum_{i=1}^z a_i B_i^{(m)}(x), \quad \forall x \in [a, b],$$

où les $B_i^{(m)}(x)$ constituent une base de l'espace spline $S_m(\xi)$ et où $z = \dim S_m(\xi)$. L'espace $S_m(\xi)$ est défini par :

$$S_m(\xi) = \{s : [a, b] \rightarrow \mathbb{R} \mid s \text{ est polynomiale par morceaux de degré } \leq m, C^{m-1} \text{ sur } [a, b], \text{ nœuds } \xi\}.$$

La formule générale de la dimension de l'espace de splines $S_m(\xi)$ sur l'intervalle $[a, b]$, avec r nœuds internes distincts et continuités C^{k_i} en chaque nœud ξ_i , est :

$$\dim S_m(\xi) = m + 1 + \sum_{i=1}^r (m - k_i)$$

Les a_i sont appelés coefficients de la spline ; ils déterminent l'amplitude de chaque fonction de base dans la combinaison qui définit la spline. Ces coefficients sont généralement déterminés en résolvant un système d'équations obtenu en imposant que la spline interpole ou approxime les points de données donnés, à l'aide de conditions de continuité et de régularité.

Ces fonctions seront utilisées pour approximer les champs vectoriels

Définition 1.3.7. Soit $m \in \mathbb{N}^*$ et $\xi = (\xi_0, \xi_1, \dots, \xi_k)$ un vecteur de nœuds

On appelle **spline d'approximation** de degré m une fonction spline utilisée pour approcher un ensemble de données sans nécessairement passer par tous les points. Contrairement à une spline d'interpolation, la spline d'approximation cherche à minimiser l'écart global (erreur quadratique) entre la courbe et les données, en assurant une certaine régularité de la fonction approchée.

Autrement, on cherche une fonction spline $f(x) = \sum_{j=1}^z a_j B_j(x)$ qui minimise l'erreur quadratique :

$$f^* = \arg \min_{a \in \mathbb{R}^z} \sum_{i=1}^n (y_i - f(x_i))^2$$

Où $\{B_j\}$ sont les fonctions de base (B-splines), et $a = (a_j)_{j=1, \dots, z}$ les coefficients. L'espace $S_m(\xi)$ est alors l'espace engendré par ces bases.

Cette approche est particulièrement utile lorsque les données sont bruitées ou comportent des incertitudes. Ses splines d'approximation sont étudiées plus en détail dans le chapitre suivant.

1.3.6 Fonction de base radiale

Définition 1.3.8. Une fonction de base radiale (RBF, Radial Basis Function) est une fonction réelle dont la valeur dépend uniquement de la distance entre un point d'entrée $x \in \mathbb{R}^d$ et un centre $c \in \mathbb{R}^d$, selon une norme donnée (souvent la norme euclidienne). Formellement, une fonction φ est dite radiale si elle s'écrit sous la forme :

$$\varphi(x) = \phi(\|x - c\|),$$

où $\phi : [0, +\infty[\rightarrow \mathbb{R}$ est une fonction réelle définie sur les distances $r = \|x - c\|$.

Les fonctions de base radiale sont utilisées notamment en interpolation, approximation de fonctions, apprentissage automatique, et résolution numérique d'équations différentielles [7].

Remarque 1.3.2. — Le terme de fonction radiale est utilisé, étant donné qu'elle ne dépend que du rayon, c'est-à-dire de la distance entre un point et le centre.

- On parle de base, car les fonctions radiales servent de base fonctionnelle dans un espace de fonctions (espace de Hilbert, espace de Sobolev, etc.), ce qui justifie leur utilité ici. Elles servent de base pour l'espace dans lequel vie le champ à estimé.

Exemples de fonctions de base radiale :

- **Gaussienne :**

$$\varphi(r) = e^{-(\epsilon r)^2},$$

où $\epsilon > 0$ est un paramètre de forme.

- **Spline en plaque mince :**

$$\varphi(r) = r^2 \ln(r),$$

définie pour $r > 0$.

1.3.7 Définitions des espaces fonctionnels[4]

Définition 1.3.9. L'espace $L^2(\mathbb{R}^d)$ est l'ensemble des fonctions mesurables $f : \mathbb{R}^d \rightarrow \mathbb{R}$ telles que

$$\|f\|_{L^2(\mathbb{R}^d)}^2 = \int_{\mathbb{R}^d} |f(x)|^2 dx < +\infty.$$

Définition 1.3.10. Pour un multi-indice $\alpha = (\alpha_1, \alpha_2, \dots, \alpha_d) \in \mathbb{N}^d$, on note

$$|\alpha| = \sum_{i=1}^d \alpha_i, \quad \text{et} \quad D^\alpha = \frac{\partial^{|\alpha|}}{\partial x_1^{\alpha_1} \partial x_2^{\alpha_2} \dots \partial x_d^{\alpha_d}}.$$

Définition 1.3.11. Soit $\Omega \subset \mathbb{R}^d$ un domaine ouvert. Pour un entier $m \geq 0$, on définit l'espace de Sobolev $H^m(\Omega)$ par :

$$H^m(\Omega) := \{u \in L^2(\Omega) : D^\alpha u \in L^2(\Omega), \forall \alpha \text{ multi-indice avec } |\alpha| \leq m\}$$

Définition 1.3.12. Pour un entier $m \geq 0$, on définit l'espace

$$D^{-m}L^2(\mathbb{R}^d) = \left\{ f : \mathbb{R}^d \rightarrow \mathbb{R} \mid D^\alpha f \in L^2(\mathbb{R}^d), \forall \alpha \in \mathbb{N}^d \text{ tel que } |\alpha| = m \right\}.$$

Autrement dit, cet espace regroupe les fonctions dont toutes les dérivées partielles d'ordre exactement m sont dans $L^2(\mathbb{R}^d)$.

Définition 1.3.13 (Espace de Beppo Levi H). L'espace de Beppo Levi d'ordre $m \in \mathbb{N}^*$ sur $\mathbb{R}^d$ est défini par

$$H = \underbrace{D^{-m}L^2(\mathbb{R}^d) \times D^{-m}L^2(\mathbb{R}^d) \times \dots \times D^{-m}L^2(\mathbb{R}^d)}_{k\text{fois}},$$

où $k \in \mathbb{N}$ est le nombre de composantes. Chaque élément $(f_1, f_2, \dots, f_k) \in H$ est une k -uplet de fonctions telles que chacune possède des dérivées partielles d'ordre m dans $L^2(\mathbb{R}^d)$.

1.3.8 Loi de von Mises

Définition 1.3.14. La loi de von Mises est une distribution de probabilité définie sur le cercle, souvent considérée comme l'analogue circulaire de la loi normale. Elle est particulièrement adaptée pour modéliser des variables angulaires, telles que des directions ou des orientations. La densité de probabilité de la loi de von Mises est donnée par :

$$f(\theta \mid \mu, \kappa) = \frac{1}{2\pi I_0(\kappa)} \exp(\kappa \cos(\theta - \mu)),$$

où :

- $\theta \in [0, 2\pi[$ est la variable aléatoire angulaire,
- $\mu \in [0, 2\pi[$ est l'angle moyen (le centre de concentration),
- $\kappa \geq 0$ est le paramètre de concentration. Plus κ est élevé, plus la distribution est concentrée autour de μ ,
- $I_0(\kappa)$ représente la fonction de Bessel modifiée d'ordre 0, définie par :

$$I_0(\kappa) = \sum_{n=0}^{\infty} \frac{1}{(n!)^2} \left(\frac{\kappa}{2}\right)^{2n}.$$

Cette loi sert à bruite nos champs vectoriels, puisque les lois usuelles comme la loi normale ou la loi exponentielle, etc ne sont pas adaptées à notre cas de figure, car il faut tenir compte de la variation angulaire, qui joue un rôle très important dans la direction du vecteur.

CHAPITRE 2

PRÉSENTATION DES DONNÉES ET MÉTHODOLOGIES

La présente section a pour objectif d'expliquer les fondements et la théorie mathématique sous-jacente aux différentes méthodes utilisées pour l'estimation des champs vectoriels tout au long de ce stage et de présenter les différents jeux de données sur lesquels le travail a été effectué. Il serait avantageux de concevoir de meilleures méthodes d'estimation des champs de vecteurs, afin de réduire le temps de traitement, les coûts de campagne et les besoins en moyens de calcul, tout en limitant le recours à de nombreuses campagnes coûteuses sur le terrain. Comme évoqués dans le chapitre précédent, les champs de vecteurs sont omniprésents dans de nombreux phénomènes naturels : courants marins, vents, mouvements atmosphériques, flux de chaleur, transports sédimentaires, courants fluviaux ou encore déformations glaciaires. Ils constituent donc un outil essentiel pour améliorer la compréhension, l'anticipation et la modélisation de processus dynamiques complexes en océanographie, météorologie, hydrologie et géophysique.

2.1 Jeux de données

On peut estimer un champ à partir de deux types de données. D'une part, des données dites directes partielles, c'est-à-dire provenant de champs déjà connus, mais considérés comme incomplets. À titre d'exemple, il peut s'agir de champs issus de modèles numériques de simulation de la circulation océanique de type copernicus ou de données de champs estimés à partir de mesurages de gradients physiques faits sur le terrain ou par satellite. D'autre part, des données indirectes par exemple, seules des images satellitaires peuvent être disponibles alors que l'on cherche à estimer un champ en 2D, tandis que le plan d'observation correspond à un écoulement 3D. On peut également posséder de données d'altimétrie fournies par des satellites. L'enjeu est alors d'extraire ou de reconstruire le champ caché derrière ces observations visuelles ce qui est particulièrement nécessaire lorsque les mesures directes sont impossibles ou difficilement accessibles, et que cela requiert de lever certaines contraintes physiques ou statistiques sur l'état de l'environnement.

2.1.1 Observation directe

S'agissant des observations directes, nous avons utilisé les données fournies par les modèles de circulation océanique disponibles via le service Copernicus Marine Environment Monitoring Service (CMEMS). Plus précisément, nous exploiterons les champs de vitesse du courant marin issus du produit `GLOBAL_ANALYSISFORECAST_PHY_001_024`, correspondant aux moyennes journalières de la circulation océanique globale qui utilise une résolution horizontale de 1/12 degré avec une projection équirectangulaire régulière en longitude et latitude. Les courants peuvent être regroupés en deux catégories, les courants de surface et les courants océaniques profonds. Les courants de surface s'écoulent jusqu'à une profondeur maximale d'environ 100 m et ont deux mécanismes principaux : les vents de surface et le flux des marées dû à l'attraction gravitationnelle du Soleil et de la Lune. Les courants océaniques profonds, quant à eux, sont créés par des différences de densité de l'eau contrôlées par des propriétés de l'eau telles que la température et la salinité[5]. Ces champs seront extraits sur une période de quelques mois pour la zone géographique d'intérêt qui est autour de l'archipel de Kerguelen situé dans les Terres australes et antarctiques françaises (TAAF) afin de servir de base à nos travaux et d'analyse puisque c'est dans ses zones que plongent les mammifères qui font l'objet de l'étude ce projet. Ce jeu de données comporte des informations à la fois spatiales (longitude, latitude, profondeur) et temporelles (dates), et comporte deux variables u_o et v_o qui sont les composantes du champ de vecteur de vitesse. La variable :

- u_o : composante **zonale** du courant (Est-Ouest).

Une valeur positive de u_o ($u_o > 0$) signifie que le courant se dirige vers l'Est, tandis qu'une valeur négative indique un courant vers l'Ouest.

- v_o : composante **méridienne** du courant (Nord-Sud).

Une valeur positive de v_o ($v_o > 0$) signifie que le courant se dirige vers le Nord, tandis qu'une valeur négative indique un courant vers le Sud.

Voici un exemple de champ de vitesse des courants océaniques.

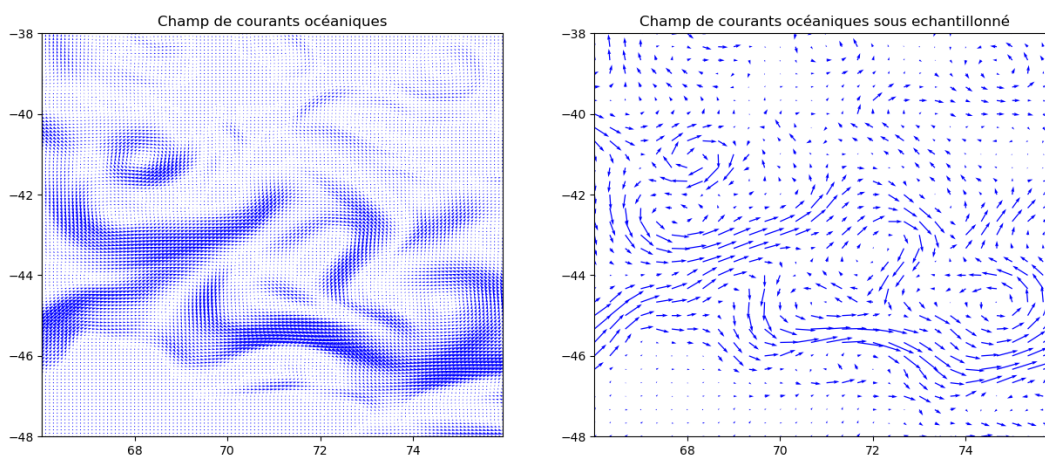


Figure 2.1 – Champ de courant récupéré sur copernicus.

Pour visualiser l'évolution complète, voir l'animation disponible [ici](#).

La figure ci-avant montre la représentation du champ de vitesse des courants océaniques observé le 7 janvier 2025 à partir des données d'écrite précédemment. Cette carte illustre l'intensité et la direction des courants marins à la surface de l'océan dans une zone géographique déterminée par les coordonnées suivantes : -38° Nord, -48°

Sud, 76° Est, et 64° Ouest. Cette zone correspond à notre zone d'intérêt.

Dans la suite, l'utilisation de ces données nécessite un échantillonnage, car la résolution initiale dont nous disposons correspond à la plus élevée existante, ce qui permet d'obtenir environ une grille équi-rectangulaire de dimensions 11, 3km et 6km . L'échantillonnage permet de vérifier si nos méthodes reconstituent correctement des champs partiellement observés, d'évaluer la résolution des modèles numériques et, par la suite, d'utiliser ces modèles pour compléter les données manquantes ou moins précises. Pour cela, deux types d'échantillonnage seront mis en œuvre, à savoir :

- **Échantillonnage systématique** : Il s'agira ici de sélectionner des points sur les axes (longitude, latitude) de manière régulière, en commençant par la plus petite valeur de l'axe concerné. On parle d'un échantillonnage systématique de pas 2 lorsque l'on sélectionne un point toutes les deux unités, et ainsi de suite.

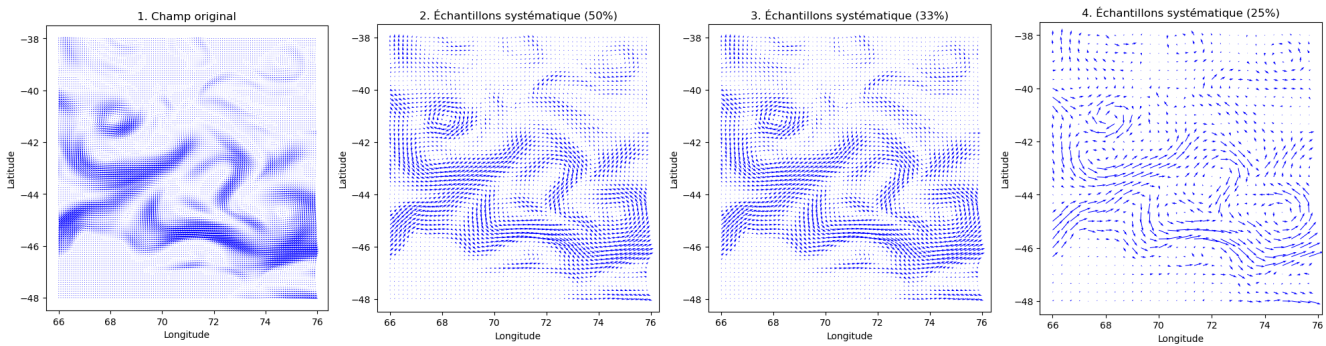


Figure 2.2 – Échantillonnage systématique.

- **Échantillonnage aléatoire** : Comme son nom l'indique, il consiste à sélectionner un ensemble de points de manière aléatoire selon une loi uniforme.

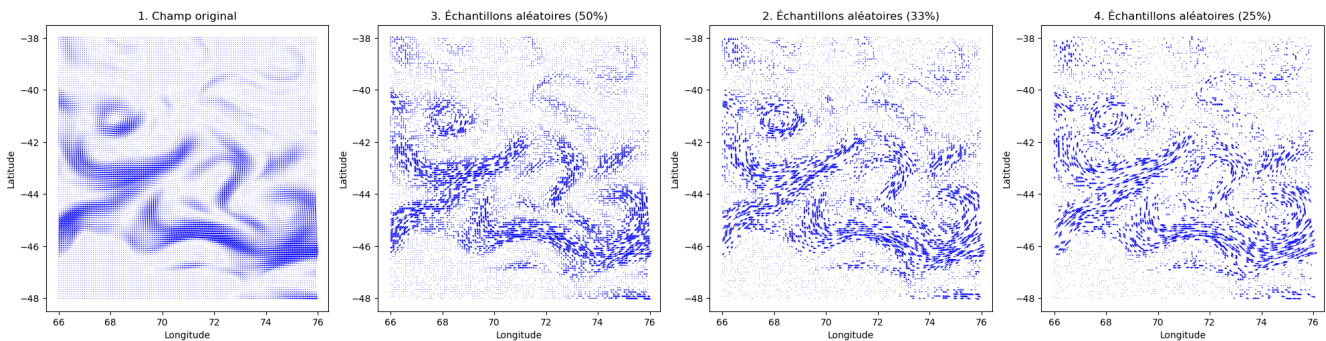


Figure 2.3 – Échantillonnage aléatoire.

Les deux images précédentes montrent les deux types d'échantillonnage appliqués aux données, en conservant respectivement 50 %, 33 % et 25 % des observations. Ce sont ces échantillons qui seront utilisés dans la suite.

2.1.2 Observations indirectes

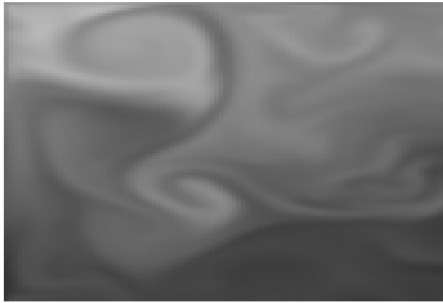
L'expression *observation indirecte* désigne ici toute type de données et/ou observations qui ne sont pas initialement un champ de vecteurs, mais qui pourront servir pour estimer ou reconstruire un champ de vecteurs. Bien que l'approche par observations indirectes n'ait pas été traitée dans le cadre de ce travail, les données associées sont présentées ici. Ces données sont principalement constituées d'images, disponibles sous différentes formes :

Données d'altimétrie océanique (Copernicus) Il s'agit d'observations indirectes obtenues à partir des satellites altimétriques. Plus précisément, ce sont des anomalies du niveau de la mer (Sea Level Anomalies, SLA) calculées par interpolation, à partir des mesures altimétriques de différentes missions satellites. Ces produits sont générés par le système DUACS (Data Unification and Altimeter Combination System) et distribués par le service Copernicus Marine (CMEMS).

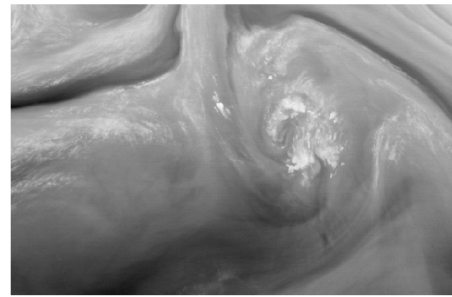
Ces données permettent également d'estimer des courants géostrophiques, dérivés des gradients de la topographie dynamique absolue (ADT). Elles sont principalement utilisées pour l'analyse et la reconstruction de la dynamique océanique à grande échelle.

Séquences d'images issues de modèles et d'observations satellitaires L'autre type de données est constitué de séries d'images utilisées dans le cadre de tests comparatifs. Ces séquences sont constituées d'images générées par des modèles de circulation océanique ainsi que d'images issues d'observations satellites (par exemple, METEOSAT).

L'objectif est ici de confronter visuellement, ou par des méthodes d'estimation, les sorties des modèles numériques aux observations réelles, afin de tester et de valider les méthodes de reconstruction de champs vectoriels à partir d'observations visuelles.



(a) OPA (modèle)



(b) METEOSAT (satellite)

Figure 2.4 – Extrait des séquences de test : à gauche, OPA ; à droite, METEOSAT. [10]

Les images présentées montrent des séquences de vapeur d'eau : à gauche, une simulation issue du modèle océanique OPA, et à droite, une observation satellitaire capturée par METEOSAT dans la bande vapeur d'eau. Ces jeux de données, bien qu'exploités ici uniquement à titre illustratif, constituent une source précieuse pour l'étude des champs vectoriels lorsqu'ils sont correctement couplés à des méthodes d'estimation appropriées.

2.2 Méthode d'interpolation

Ici, on suppose que nous connaissons le champ vectoriel en un certain nombre de points appelés points de contrôle ou points de mesure. Plus précisément, en chaque point $\mathbf{x}_i = (x_i, y_i)$, on connaît la valeur associée $\mathbf{w}_i = (u_i, v_i)$.

L'objectif est donc de trouver un champ vectoriel $\mathbf{w}(x, y)$ sur tout le domaine, de manière à ce qu'il prenne exactement ou presque ces valeurs aux points connus :

$$\mathbf{w}(\mathbf{x}_i) = \mathbf{w}_i \text{ ou bien } \mathbf{w}(\mathbf{x}_i) \approx \mathbf{w}_i \quad \text{pour } i = 1, \dots, n.$$

De façon naturelle, puisque nous disposons des points de contrôle, pour obtenir la valeur du champ en point

que nous ne connaissons pas, il suffira juste de faire une interpolation ce qui permettra de construire des fonctions qui prennent exactement les valeurs aux points connus. En dimension 2, l'interpolation peut se faire en utilisant plusieurs méthodes :

2.2.1 Interpolation bilinéaire

Méthode des rectangles : L'interpolation bilinéaire est une méthode simple et efficace permettant d'estimer la valeur d'une fonction $f(x, y)$ en un point inconnu à partir des valeurs connues aux quatre sommets d'un rectangle de la grille. Cette approche repose sur une approximation affine de la fonction sous la forme suivante :

$$f(x, y) = a x + b y + c x y + d$$

où a, b, c, d sont déterminées à partir des 4 valeurs connues.

Si l'on souhaite améliorer l'approximation, notamment lorsque la fonction à estimer présente des variations plus complexes, on peut utiliser un modèle polynomial de degré supérieur. De manière générale, l'interpolation polynomiale d'ordre p en deux dimensions s'écrit sous la forme :

$$f(x, y) = \sum_{i=0}^p a_i x^i y^{p-i}$$

où a_i sont les coefficients inconnus, et p désigne le degré du polynôme. Le choix du degré dépend du compromis recherché entre la précision de l'approximation et la stabilité numérique de l'interpolation.

En effet, soit un point (x, y) à interpoler, encadré par les quatre sommets (x_1, y_1) , (x_2, y_1) , (x_1, y_2) et (x_2, y_2) de la grille, auxquels sont associées les valeurs $f(x_i, y_j)$. Les quatre conditions d'interpolation donnent le système suivant :

$$\begin{cases} f(x_1, y_1) = a x_1 + b y_1 + c x_1 y_1 + d, \\ f(x_2, y_1) = a x_2 + b y_1 + c x_2 y_1 + d, \\ f(x_1, y_2) = a x_1 + b y_2 + c x_1 y_2 + d, \\ f(x_2, y_2) = a x_2 + b y_2 + c x_2 y_2 + d. \end{cases}$$

On introduit.

$$dx = x - x_1, \quad dy = y - y_1,$$

ainsi que

$$\Delta x = x_2 - x_1, \quad \Delta y = y_2 - y_1.$$

Le modèle se réécrit alors sous la forme simplifiée.

$$f(dx, dy) = a dx + b dy + c dx dy + d.$$

On définit les accroissements :

$$\Delta f_x = f(x_2, y_1) - f(x_1, y_1), \quad \Delta f_y = f(x_1, y_2) - f(x_1, y_1),$$

$$\Delta f_{xy} = f(x_1, y_1) + f(x_2, y_2) - f(x_2, y_1) - f(x_1, y_2).$$

La solution de l'interpolation s'écrit alors :

$$f(x, y) = f(x_1, y_1) + \Delta f_x \frac{dx}{\Delta x} + \Delta f_y \frac{dy}{\Delta y} + \Delta f_{xy} \frac{dx}{\Delta x} \frac{dy}{\Delta y}.$$

Cette approche est appliquée séparément à chaque composante du champ, permettant ainsi d'interpoler indépendamment chaque dimension. Cette méthode est appliquée dans le cas de l'échantillonnage systématique comme le montre la figure suivante :

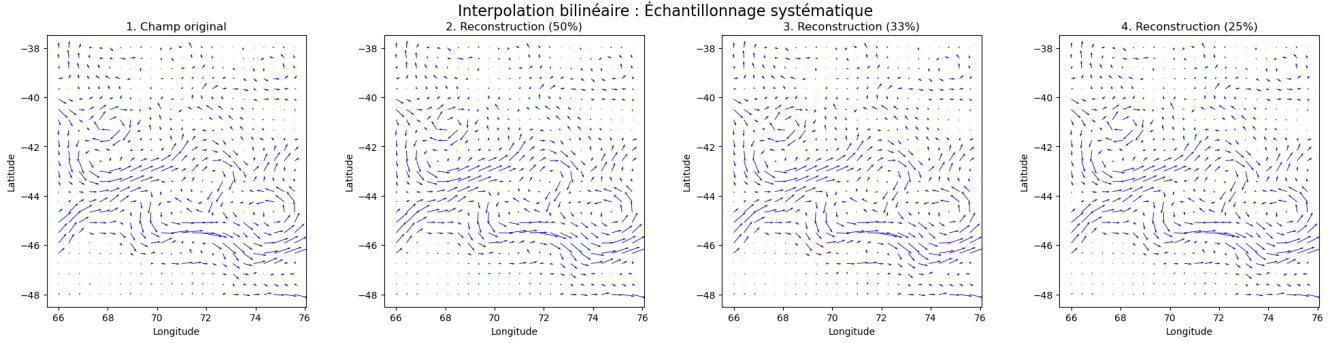


Figure 2.5 – Échantillonnage systématique-Interpolation bilinéaire.

Cette figure illustre un cas d'application de la méthode des rectangles. On y observe quatre représentations : le champ initial complet, puis les champs reconstruits à partir d'un sous-échantillonnage conservant respectivement 50 %, 33 % et 25 % des points de mesure du champ initial.

Méthode des barycentres : La condition d'encadrement par un rectangle n'est pas toujours réalisable, en particulier lorsque les points d'échantillonnage ne sont pas distribués selon une grille régulière. Une alternative est l'interpolation par barycentres. L'idée est la suivante : $\triangleright$ on construit la triangulation de Delaunay du nuage de points connus. Autrement dit, on relie les points de telle sorte qu'ils forment des triangles respectant la propriété suivante : *le cercle circonscrit à chaque triangle ne contient aucun autre point à l'intérieur. Cette triangulation fournit un maillage régulier et bien conditionné, évitant les triangles trop plats.*

$\triangleright$ pour un point $\mathbf{x}$ à interpoler, on recherche le triangle de la triangulation qui le contient ;

$\triangleright$ les coordonnées barycentriques $(\lambda_1, \lambda_2, \lambda_3)$ du point par rapport aux sommets $(\mathbf{x}_1, \mathbf{x}_2, \mathbf{x}_3)$ du triangle sont calculées. Avec :

$$\lambda_1 = \frac{\mathcal{A}(\mathbf{x}, \mathbf{x}_2, \mathbf{x}_3)}{\mathcal{A}(\mathbf{x}_1, \mathbf{x}_2, \mathbf{x}_3)}, \quad \lambda_2 = \frac{\mathcal{A}(\mathbf{x}, \mathbf{x}_3, \mathbf{x}_1)}{\mathcal{A}(\mathbf{x}_1, \mathbf{x}_2, \mathbf{x}_3)}, \quad \lambda_3 = \frac{\mathcal{A}(\mathbf{x}, \mathbf{x}_1, \mathbf{x}_2)}{\mathcal{A}(\mathbf{x}_1, \mathbf{x}_2, \mathbf{x}_3)},$$

où $\mathcal{A}(\mathbf{a}, \mathbf{b}, \mathbf{c})$ désigne l'aire du triangle formé par les trois points $\mathbf{a}, \mathbf{b}, \mathbf{c}$.

$\triangleright$ La valeur interpolée s'écrit alors comme une combinaison convexe :

$$f(\mathbf{x}) = \lambda_1 f(\mathbf{x}_1) + \lambda_2 f(\mathbf{x}_2) + \lambda_3 f(\mathbf{x}_3).$$

Cette approche, appliquée séparément à chaque composante du champ afin d'interpoler indépendamment chaque dimension, est illustrée ici dans le cas d'un échantillonnage aléatoire (voir figure suivante).

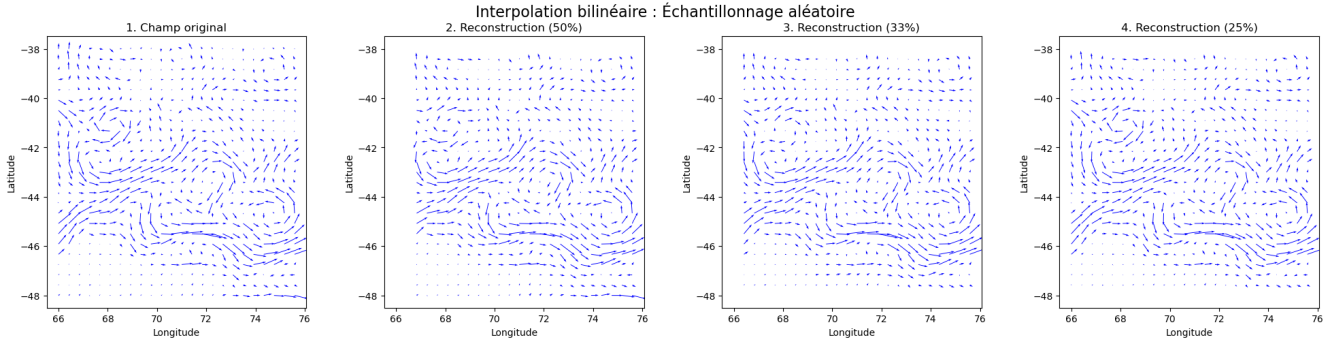


Figure 2.6 – Sous-échantillonnage aléatoire - Méthode des barycentres.

Cette figure illustre quant à elle la méthode des barycentres, présentée de manière analogue. Elle comporte également le champ complet ainsi que les reconstructions issues des trois taux de sous-échantillonnage. »

Dans les deux cas, la reconstruction apparaît visuellement presque parfaite, même lorsque seuls 25 % des points de mesure sont conservés. Les structures et caractéristiques principales du champ initial sont correctement préservées.

2.2.2 Interpolation avec B-splines

Ici, c'est une méthode basée sur des splines, des fonctions qui permettent de reconstruire des formes lisses à partir de quelques points de contrôle. L'idée est de reconstruire le champ $f(\mathbf{x})$ à partir de valeurs connues sur certains points. On exprime alors $f(\mathbf{x})$ comme une combinaison de fonctions de base :

$$f(\mathbf{x}) = \sum_{i=1}^n a_i B_i^{(m)}(x) B_i^{(m)}(y)$$

où :

- $B_i^{(m)}(x)$ et $B_i^{(m)}(y)$ sont les fonctions de base splines d'ordre m dans les directions x et y .
- a_i sont les coefficients à déterminer via les conditions d'interpolation.

En effet, en imposant les conditions d'interpolation :

$$f(\mathbf{x}_i) = f_i \Rightarrow \sum_{i=1}^n a_i B_i^{(m)}(x_i) B_i^{(m)}(y_i) = f_i, \quad i = 1, \dots, n.$$

On ajoute aussi des conditions supplémentaires sur les dérivées aux extrémités, ce qui peut modifier légèrement le système.

De façon analogue, cette approche est appliquée séparément à chaque composante du champ, permettant ainsi d'interpoler indépendamment chaque dimension.

Les figures suivantes montrent l'application de cette méthode sur notre champ de référence dans différents cas de figure.

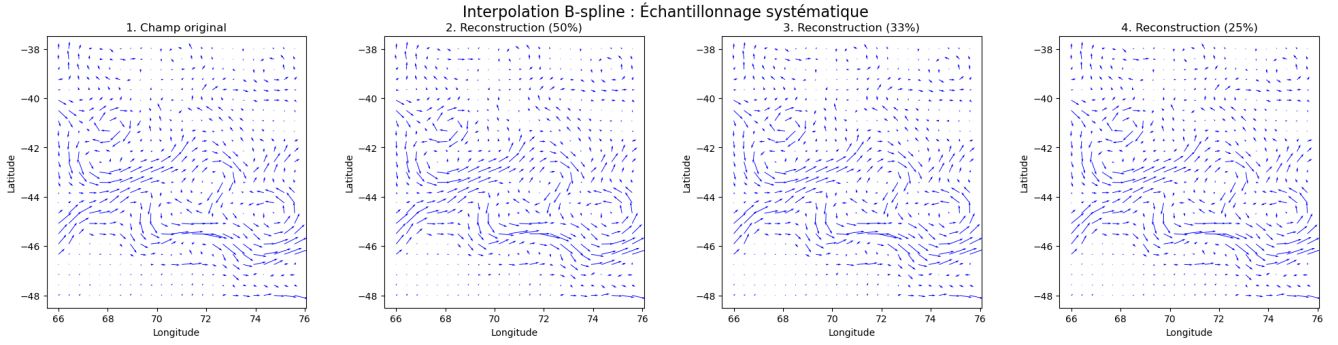


Figure 2.7 – Échantillonnage systématique - Interpolation avec B-spline.

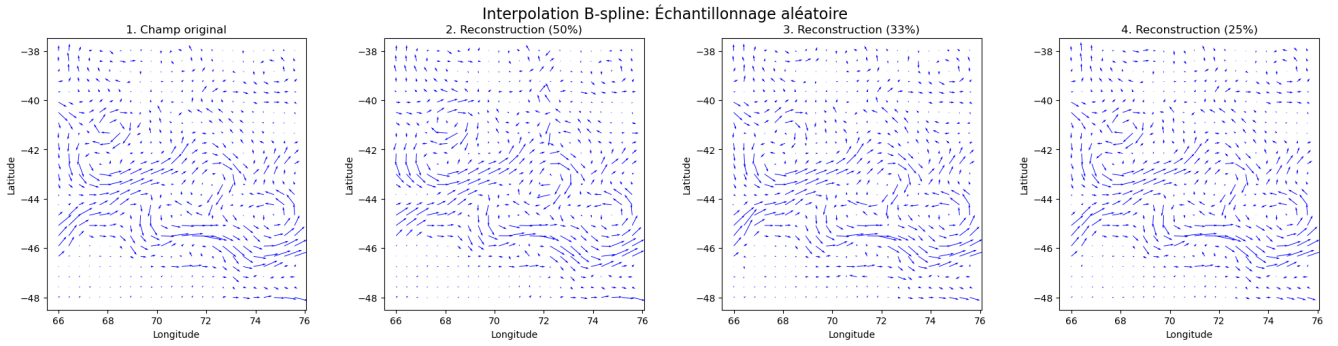


Figure 2.8 – Échantillonnage aléatoire - Interpolation avec B-spline.

Ces deux figures illustrent l'application de deux méthodes différentes d'échantillonnage, visant à réduire la taille des données, selon les deux types d'échantillonnage étudiés. Dans chaque figure, quatre représentations sont présentées comme précédemment, la reconstruction étant réalisée par interpolation à l'aide des B-splines ici. Les résultats sont similaires à celui obtenu précédemment, la reconstruction est presque parfaite pour les trois taux d'échantillonnage.

Cependant, ces approches présentent certaines limites. Elles ne tiennent pas compte des relations potentielles entre les deux composantes du vecteur (u, v) , notamment les contraintes physiques qui régissent l'écoulement ou le phénomène étudié, c'est pour cela L. Amodéi et M. N. Benbourhim [1] propose une approximation spline spécifique aux champs vectoriels, qui tient compte de la cohérence globale du champ et des éventuelles relations physiques entre ses composantes par le biais de la norme div-curl.

2.2.3 Méthode de L. Amodéi 1991 [1]

On cherche un champ vectoriel $\mathbf{w} : \mathbb{R}^2 \rightarrow \mathbb{R}^2$ tel que :

$$\mathbf{w} = \arg \min_{\substack{\mathbf{v} \in H \\ \mathbf{w}(\mathbf{x}_i) = \mathbf{w}_i}} \int_{\Omega} \alpha \|\nabla \operatorname{div} \mathbf{w}\|^2 + \beta \|\nabla \operatorname{rot} \mathbf{w}\|^2 dx dy$$

où :

- H est l'espace de Beppo Levi $D^{-2}L^2(\mathbb{R}^2) \times D^{-2}L^2(\mathbb{R}^2)$ avec $D^{-2}L^2(\mathbb{R}^2) = \left\{ f \mid \frac{\partial^2 f}{\partial x^\alpha \partial y^{2-\alpha}} \in L^2(\mathbb{R}^2), \quad \forall \alpha = 0, 1, 2 \right\}$;
- $\mathbf{w}_i$ sont les valeurs connues aux points $\mathbf{x}_i$;

- $\alpha, \beta \geq 0$ sont deux paramètres positifs permettant de pondérer respectivement l'importance de la divergence et de la rotation ;

Le problème est posé dans un cadre fonctionnel approprié (espace de Beppo-Levi), de sorte que les dérivées intervenant dans l'énergie soient de carrés intégrables, même si les composantes u et v de $\mathbf{w}$ ne sont pas nécessairement dans L^2 . Par un théorème de représentation pour les splines vectorielles (voir [1]), la solution interpolante qui minimise :

$$\min_{\mathbf{w} \in H} \int_{\Omega} \alpha \|\nabla \operatorname{div} \mathbf{w}\|^2 + \beta \|\nabla \operatorname{rot} \mathbf{w}\|^2 dx dy$$

sous les contraintes $\mathbf{w}(\mathbf{x}_i) = \mathbf{w}_i$ s'écrit comme une somme de noyaux radiaux translatés à laquelle s'ajoute un terme polynomial. L'existence et l'unicité découlent du cadre variationnel et des conditions d'orthogonalité associées au terme polynomial, tandis que les poids α et β règlent le compromis entre les parts divergence et rotationnel. Cette solution interpolante s'écrit :

$$\begin{cases} u(\mathbf{x}) = p(\mathbf{x}) + \sum_{i=1}^n \left[a_i \left(\frac{1}{\alpha} \phi_{xx}(\mathbf{x} - \mathbf{x}_i) + \frac{1}{\beta} \phi_{yy}(\mathbf{x} - \mathbf{x}_i) \right) + b_i \left(\frac{1}{\alpha} - \frac{1}{\beta} \right) \phi_{xy}(\mathbf{x} - \mathbf{x}_i) \right], \\ v(\mathbf{x}) = q(\mathbf{x}) + \sum_{i=1}^n \left[a_i \left(\frac{1}{\alpha} - \frac{1}{\beta} \right) \phi_{xy}(\mathbf{x} - \mathbf{x}_i) + b_i \left(\frac{1}{\alpha} \phi_{yy}(\mathbf{x} - \mathbf{x}_i) + \frac{1}{\beta} \phi_{xx}(\mathbf{x} - \mathbf{x}_i) \right) \right]. \end{cases}$$

avec $\phi(\mathbf{x}) = (128\pi)^{-1} \|\mathbf{x}\|^4 \log \|\mathbf{x}\|$ et p, q polynômes de degré 1. Et les coefficients sont obtenus en résolvant le système $(2n+6)(2n+6)$ suivant :

$$\begin{bmatrix} M & P \\ P^t & 0 \end{bmatrix} \begin{bmatrix} a \\ b \\ \alpha \\ \beta \end{bmatrix} = \begin{bmatrix} u \\ v \\ 0_6 \end{bmatrix}, \quad (2.1)$$

Avec

$$M_{ij} = \frac{1}{\alpha} \begin{bmatrix} \phi_{xx}(\mathbf{x}_i - \mathbf{x}_j) & \phi_{xy}(\mathbf{x}_i - \mathbf{x}_j) \\ \phi_{xy}(\mathbf{x}_i - \mathbf{x}_j) & \phi_{yy}(\mathbf{x}_i - \mathbf{x}_j) \end{bmatrix} + \frac{1}{\beta} \begin{bmatrix} \phi_{xx}(\mathbf{x}_i - \mathbf{x}_j) & -\phi_{xy}(\mathbf{x}_i - \mathbf{x}_j) \\ -\phi_{xy}(\mathbf{x}_i - \mathbf{x}_j) & \phi_{yy}(\mathbf{x}_i - \mathbf{x}_j) \end{bmatrix}$$

$$P = \begin{bmatrix} \begin{bmatrix} 1 & x_1 & y_1 \\ \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot \\ 1 & x_n & y_n \end{bmatrix} & 0 \\ 0 & \begin{bmatrix} 1 & x_1 & y_1 \\ \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot \\ 1 & x_n & y_n \end{bmatrix} \end{bmatrix}$$

L'équation 2.1 admet une solution unique. En effet, grâce à la condition d'orthogonalité

$$P^T \begin{bmatrix} a \\ b \end{bmatrix} = 0$$

qui élimine la composante polynomiale. Sous les hypothèses usuelles (i) P de plein rang et (ii) M est symétrique et définie positive [1], la matrice

$$\begin{bmatrix} M & P \\ P^T & 0 \end{bmatrix}$$

est non singulière. L'orthogonalité joue ainsi un rôle important. Elle lève l'ambiguïté due au noyau polynomial et avec (i)-(ii), assure l'unicité et la stabilité de la solution.

Ce critère cherche à obtenir un champ qui s'ajuste aux observations tout en contrôlant ses comportements divergents ou tournants, ce qui est essentiel pour modéliser des écoulements fluides cohérents physiquement. Cette approche permet de prendre en compte explicitement les propriétés physiques du phénomène, en ajustant le poids relatif entre divergence et rotation. Cette méthode a été appliquée à différents cas de α et β , mais nous présentons ici le cas des paramètres $\alpha = 0,9$ et $\beta = 0,1$, c'est-à-dire que l'on donne plus de poids à l'effet de divergence (éloignement/rapprochement) qu'à l'effet du rotationnel (tournis). Les résultats obtenus dans ce cas sont illustrés par les figures suivantes :

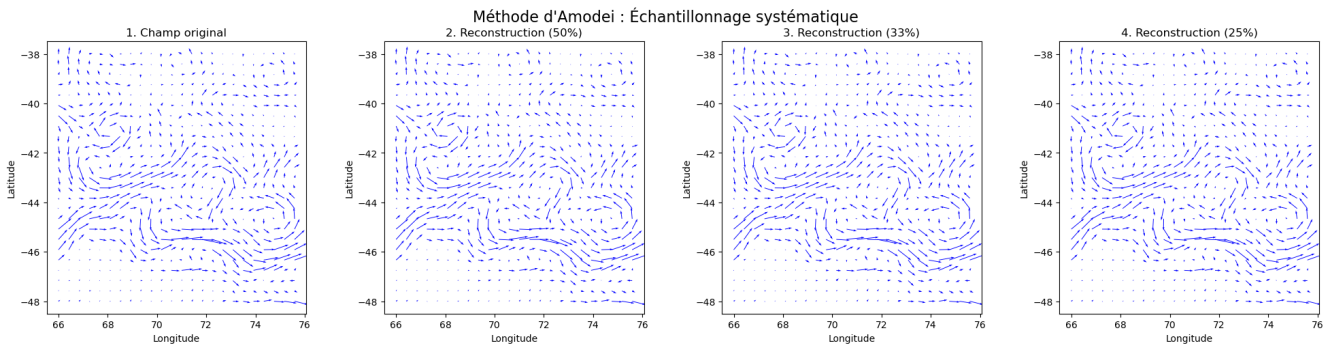


Figure 2.9 – Échantillonnage systématique - Méthode d'Amodéi.

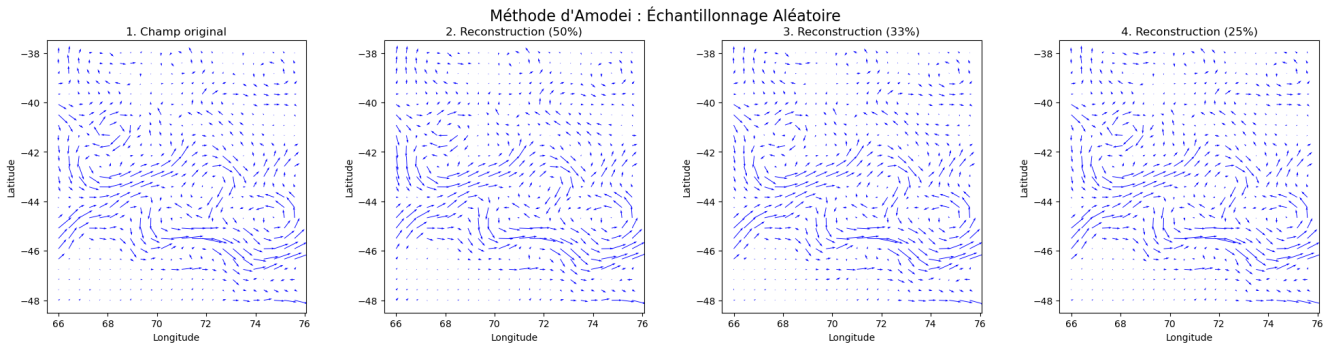


Figure 2.10 – Échantillonnage aléatoire - Méthode d'Amodéi.

Les figures 2.9 et 2.10 montrent deux façons d'échantillonner pour réduire la quantité de données. Pour chaque figure, on affiche quatre vues, comme expliqué plus haut. La reconstruction est faite par interpolation avec la

méthode d'interpolation d'Amodéi. Les résultats pour les trois taux d'échantillonnage montrent une reconstruction est presque parfaite visuellement. Des résultats complémentaires pour d'autres valeurs des paramètres α et β figurent en annexe.

2.3 Méthodes d'approximation

Pour toutes les méthodes présentées lorsque nous disposons des observations, en dehors de la grille de données, il devient difficile d'obtenir une approximation fiable, car l'estimation repose uniquement sur les points disponibles, ne généralise pas bien dans les zones mal couvertes et est très sensible au bruit. L'interpolation a donc tendance à simplement suivre les données observées sans exploiter pleinement les structures sous-jacentes du champ. Pour cela, dans ce travail, nous avons privilégié des approches d'approximation plutôt que d'interpolation, afin d'obtenir des modèles qui sont moins sensibles au bruit, moins dépendants des observations ponctuelles, capables de mieux se généraliser en dehors du domaine étudié. C'est dans cette optique que le problème suivant a été formulé :

On cherche à un champ vectoriel $\mathbf{w}$ tel que :

$$\mathbf{w}_i = \mathbf{w}(\mathbf{x}_i) + \epsilon_i, i = 1, \dots, n$$

ou $\epsilon_i = (\epsilon_i^{(1)}, \epsilon_i^{(2)})^T, i = 1, \dots, n$ est un bruit. Autrement dit, en séparant les deux composantes :

$$\begin{cases} u_i = u(\mathbf{x}_i) + \epsilon_i^{(1)} \\ v_i = v(\mathbf{x}_i) + \epsilon_i^{(2)} \end{cases} \quad \text{pour } i = 1, \dots, n$$

Une approximation du champ selon les moindres carrés est :

$$\mathbf{w} = \arg \min_{\mathbf{w} \in H^2(\Omega)^2} \sum_{i=1}^n \|\mathbf{w}(\mathbf{x}_i) - \mathbf{w}_i\|^2, \quad (2.2)$$

Et nous avons donc :

$$\arg \min_{\mathbf{w} \in H^2(\Omega)^2} \sum_{i=1}^n \|\mathbf{w}(\mathbf{x}_i) - \mathbf{w}_i\|^2 \Leftrightarrow \arg \min_{u \in H^2(\Omega)} \sum_{i=1}^n (u(\mathbf{x}_i) - u_i)^2 + \arg \min_{v \in H^2(\Omega)} \sum_{i=1}^n (v(\mathbf{x}_i) - v_i)^2$$

Il suffit donc de régler le problème pour une fonction $f : \mathbb{R}^2$ dans $\mathbb{R}$ et puis on l'applique sur chaque composante de $\mathbf{w}$

Ce problème peut être abordé à l'aide de différentes méthodes en estimant chaque composante, telle que la régression linéaire, la régression à noyau, etc. Dans ce travail, nous nous concentrons spécifiquement sur les méthodes des splines plaque mince, car elles sont plus adaptées au problème parce qu'elles offrent un compromis naturel entre fidélité aux données et régularité du champ, s'adaptent à des points irréguliers et assurent une reconstruction lisse et stable, même en présence de bruit.

2.3.1 Splines plaque mince

Dans le cadre des splines, on ajoute un terme de régularisation au problème d'interpolation afin de lisser la solution. Pour les points de mesure $(\mathbf{x}_i, f_i)_{i=1, \dots, n}$ avec f_i la valeur de la fonction à estimer au point $\mathbf{x}_i$. Le problème devient alors un compromis entre l'ajustement aux données et la régularité de la fonction, exprimé par la minimisation de la fonctionnelle suivante :

$$\mathcal{E}_\lambda(f) = \sum_{i=1}^n \|f_i - f(\mathbf{x}_i)\|^2 + \lambda \int_{\mathbb{R}^2} \left[\left(\frac{\partial^2 f}{\partial x^2} \right)^2 + 2 \left(\frac{\partial^2 f}{\partial x \partial y} \right)^2 + \left(\frac{\partial^2 f}{\partial y^2} \right)^2 \right] dx dy,$$

où $\lambda > 0$ est un paramètre de lissage contrôlant la rigidité de la déformation.

De façon générale, si $f : \mathbb{R}^d \rightarrow \mathbb{R}$

Le problème consiste à minimiser :

$$\mathcal{E}_\lambda(f) = \sum_{i=1}^n (f_i - f(\mathbf{x}_i))^2 + \lambda \int_{\mathbb{R}^d} \sum_{|\alpha|=m} \binom{m}{\alpha} (D^\alpha f(\mathbf{x}))^2 d\mathbf{x}, \quad (2.3)$$

où :

- $m \in \mathbb{N}$ est l'ordre de régularité requis (typiquement $m = 2$),
- $\alpha = (\alpha_1, \dots, \alpha_d)$ est un multi-indice avec $|\alpha| = \sum_{j=1}^d \alpha_j = m$,
- $D^\alpha f = \frac{\partial^{|\alpha|} f}{\partial x_1^{\alpha_1} \dots \partial x_d^{\alpha_d}}$,
- $\lambda > 0$ est un paramètre de lissage qui contrôle le compromis entre l'ajustement aux données et la régularité de la solution,
- $\binom{m}{\alpha} = \frac{m!}{\alpha_1! \alpha_2! \dots \alpha_d!}$ est le coefficient multinomial.

Une solution générale élégante à ce problème a été proposée par Duchon (1977) [12], sous certaines conditions sur m et d , dans le cadre des splines de plaque mince généralisées, permettant de traiter le cas de dimension arbitraire d à l'aide de techniques variationnelles adaptées. Si m est choisi de manière à ce que $2m > d$, et en fait, pour obtenir des résultats « visuellement lisses », il est préférable que $2m > d + 1$. Sous la première de ces contraintes, la solution de $\hat{f}$ qui minimise 2.3,

$$\hat{f}(\mathbf{x}) = \sum_{i=1}^n \delta_i \eta_{md}(\|\mathbf{x} - \mathbf{x}_i\|) + \sum_{j=1}^M \alpha_j \phi_j(\mathbf{x})$$

où δ et α sont des vecteurs de coefficients à estimer, δ étant soumis à la contrainte linéaire $\mathbf{T}^T \delta = 0$, où $T_{ij} = \phi_j(x_i)$. Les $M = \binom{m+d-1}{d}$ fonctions ϕ_j sont des polynômes linéairement indépendants engendrant l'espace des polynômes de degré inférieur à m . Les ϕ_i engendrent l'espace des fonctions pour lesquelles J_{md} (terme de régularisation) est nul, c'est-à-dire le « noyau » de J_{md} : ces fonctions sont considérées comme étant « complètement lisses ».

Les fonctions de base restantes utilisées sont définies par :

$$\eta_{md}(r) = \begin{cases} \frac{(-1)^{m+1+d/2}}{2^{2m-1} \pi^{d/2} (m-d/2)!} r^{2m-d} \log(r), & \text{si } d \text{ est pair} \\ \frac{\Gamma(d/2-m)}{2^{2m} \pi^{d/2} (m-1)!} r^{2m-d}, & \text{si } d \text{ impair} \end{cases}$$

En définissant maintenant la matrice Φ par $\Phi_{ij} = \eta_{md}(\|\mathbf{x}_i - \mathbf{x}_j\|)$, le problème de l'ajustement des splines

plaque mince devient :

$$\min \|\mathbf{y} - \Phi\delta - T\alpha\|^2 + \lambda\delta^T\Phi\delta \quad \text{sous la contrainte } T^T\delta = 0, \text{ avec } \mathbf{y} = (f_1, \dots, f_n) \quad (2.4)$$

La spline plaque mince $\hat{f}$, est en quelque sorte un lisseur idéal, elle a été construite en définissant précisément ce que signifie être lisse, combien de poids attribuer aux objectifs contradictoires d'ajustement des données et de lissage de $\hat{f}$, et en trouvant la fonction qui satisfait le mieux l'objectif de lissage résultant.

Posons :

$$\mathcal{J}(\delta, \alpha) = \|\mathbf{y} - \Phi\delta - T\alpha\|^2 + \lambda\delta^T\Phi\delta + \gamma^T(T^T\delta)$$

On annule les dérivées par rapport à δ et α .

Le gradient de la fonction $\mathcal{J}$ par rapport à δ est donné par :

$$\nabla_{\delta}\mathcal{E}(\delta) = -2\Phi^T(\mathbf{y} - \Phi\delta - T\alpha) + 2\lambda\Phi\delta.$$

Le gradient de la fonction $\mathcal{J}$ par rapport à α est donné par :

$$-2T^T(\mathbf{y} - \Phi\delta - T\alpha) = 0$$

Contrainte :

$$T^T\delta = 0$$

On obtient donc le système suivant, en utilisant le fait que Φ est symétrique définie positive :

$$\begin{cases} (\Phi + \lambda I)\delta + T\alpha = \mathbf{y} \\ T^T\delta = 0 \end{cases}$$

Ce qui est équivalent à :

$$\begin{bmatrix} \Phi + \lambda I & T \\ T^T & 0 \end{bmatrix} \begin{bmatrix} \delta \\ \alpha \end{bmatrix} = \begin{bmatrix} \mathbf{y} \\ 0 \end{bmatrix}$$

La solution est obtenue via la résolution du système linéaire.

Cette méthode a été appliquée au champ étudié afin d'évaluer ses performances de reconstruction sous différents types et taux d'échantillonnage. Les figures suivantes présentent les résultats visuels obtenus dans plusieurs cas.

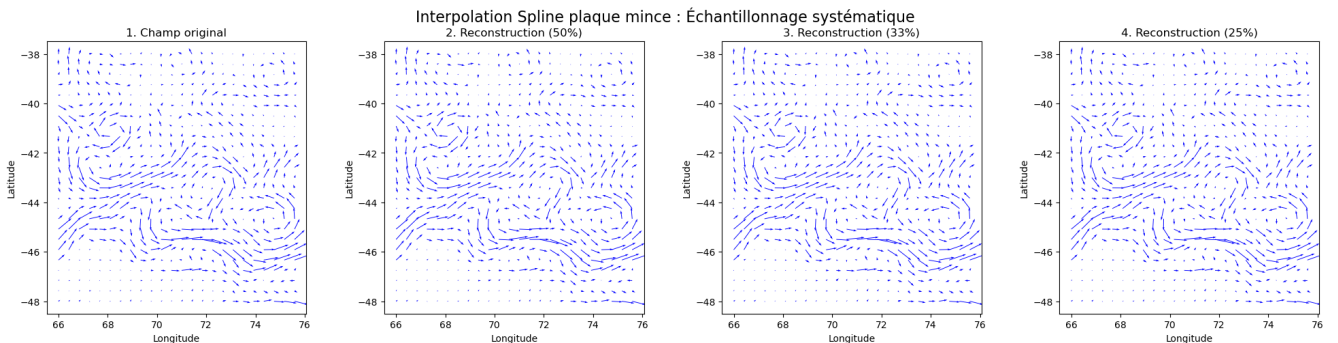


Figure 2.11 – Échantillonnage systématique - Interpolation spline plaque mince.

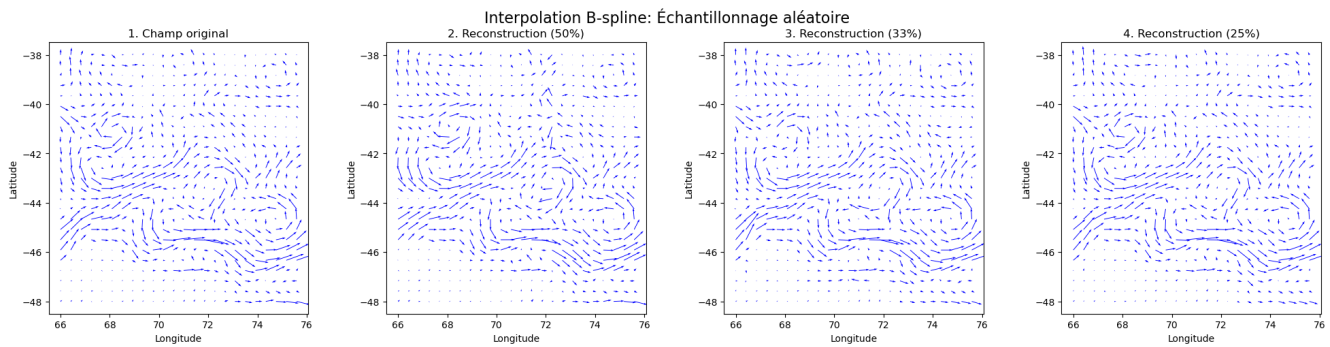


Figure 2.12 – Échantillonnage aléatoire - Interpolation spline plaque mince.

Les figures 2.9 et 2.10 présentent les résultats obtenus avec l'interpolation spline plaque mince appliquée respectivement à un échantillonnage systématique et à un échantillonnage aléatoire. Dans les deux cas, la reconstruction visuelle du champ à partir des points conservés montre une fidélité remarquable par rapport au champ initial, même lorsque le taux d'échantillonnage est réduit à 25 %. Les structures principales et les variations spatiales du champ sont correctement restituées, confirmant l'efficacité de la méthode d'interpolation utilisée. On note toutefois que l'échantillonnage systématique (figure 2.9) tend à produire une reconstruction légèrement plus régulière, tandis que l'échantillonnage aléatoire (figure 2.10) offre une répartition plus homogène des petites erreurs résiduelles.

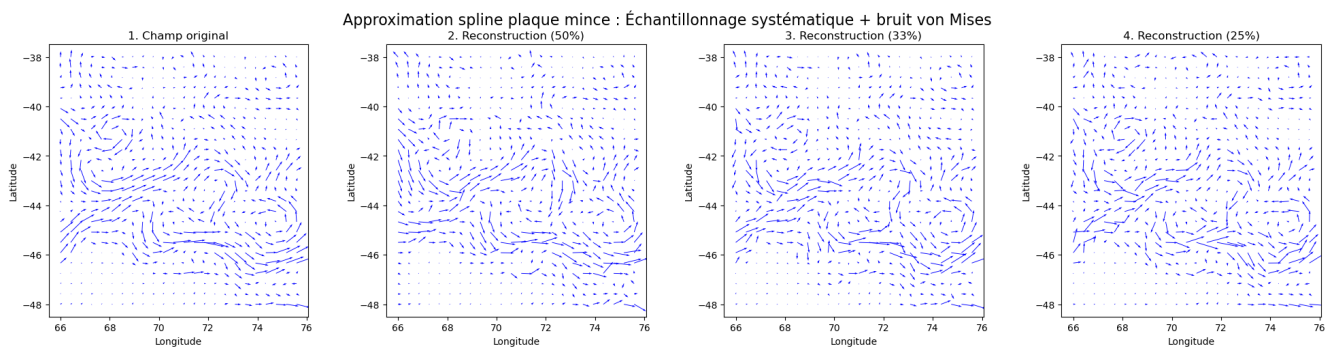


Figure 2.13 – Échantillonnage systématique - Approximation spline plaque mince.

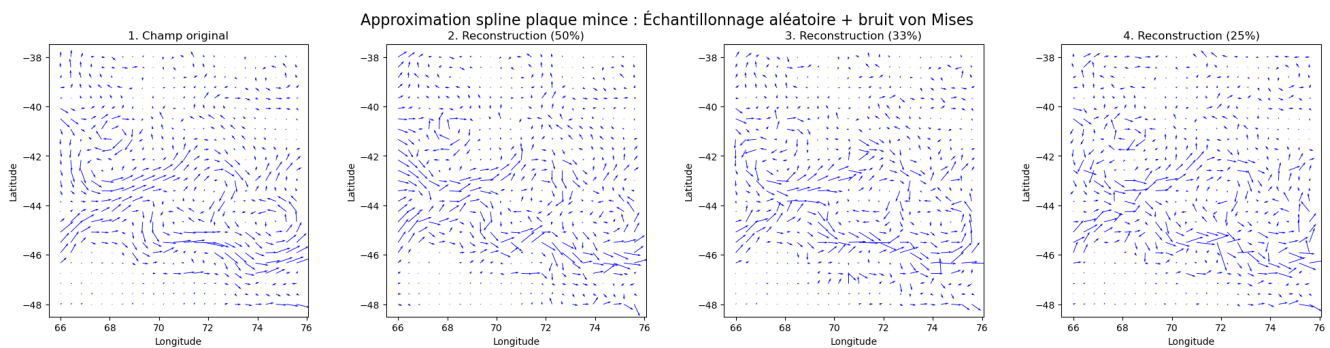


Figure 2.14 – Échantillonnage aléatoire - Approximation spline plaque mince.

Les figures 2.11 et 2.12 illustrent l'approximation spline plaque mince appliquée respectivement à un échantillonnage systématique et à un échantillonnage aléatoire, sur un champ échantillonné auquel un bruit a été ajouté.

La régularisation utilisée permet de lisser les reconstructions et de réduire l'influence du bruit. L'échantillonnage systématique conserve une meilleure régularité et une restitution plus fidèle, tandis que l'échantillonnage aléatoire présente davantage d'irrégularités liées au bruit résiduel.

2.3.2 Algorithme d'estimation du champ vectoriel

Soit $\mathbf{w} = (u, v)$ un champ vectoriel défini sur un domaine Ω . L'estimation de ce champ consiste à déterminer séparément ses composantes scalaires u et v à partir des données disponibles.

L'algorithme général d'estimation peut se décrire de la manière suivante :

Algorithm 1 Estimation séparée des composantes u et v

;

Require : Données d'observation partielles ou bruitées sur Ω ;

Require : La grille complète de Ω sur laquelle on veut estimer le champ;

- 1: Construire $\hat{u}$ et $\hat{v}$ à partir des données;
 - 2: **for** chaque point de la grille **do**
 - 3: Utiliser $\hat{u}$ et $\hat{v}$ pour prédire la valeur du champ;
 - 4: **end for**
 - 5: Combiner les composantes estimées pour reconstruire le champ $\mathbf{w}$;
 - 6: Évaluer la qualité de l'estimation à l'aide de critères appropriés;
-

2.4 Méthode tenant compte des propriétés physiques

Toutes les méthodes d'approximation utilisées jusqu'ici ne tiennent pas compte des propriétés physiques de l'écoulement ou du phénomène, mais également nous estimons les deux composantes de façon séparée. Alors pour pouvoir tenir compte de ces deux paramètres, nous posons le problème de minimisation avec une pénalisation liée à la régularité div-curl.

Donc on cherche à approximer un champ vectoriel $\mathbf{w} : \mathbb{R}^2 \rightarrow \mathbb{R}^2$ tel que :

$$\mathbf{w} = \arg \min \sum_{i=1}^n \|\mathbf{w}(\mathbf{x}_i) - \mathbf{w}_i\|^2 + \int_{\Omega} \alpha \|\nabla \operatorname{div} \mathbf{w}\|^2 + \beta \|\nabla \operatorname{rot} \mathbf{w}\|^2 dx dy, \quad (2.5)$$

- Le premier terme mesure l'erreur d'ajustement entre le champ estimé $\mathbf{w}$ et les données observées $\mathbf{w}_i$ aux points $\mathbf{x}_i$.
- α et β des paramètres.
- L'intégrale assure la régularité globale du champ estimé.

Pour assurer que le thème de régularité soit défini, on imposera que $\mathbf{w} \in H^2(\Omega)^2$.

Il a été prouvé dans [10] que :

$$\|w\|_d = \|Q(D)\mathbf{w}\|^2$$

Avec

$$Q(D) = \begin{pmatrix} \sqrt{\alpha} \partial_{xx} + \sqrt{\beta} \partial_{yy} & (\sqrt{\alpha} - \sqrt{\beta}) \partial_{xy} \\ (\sqrt{\alpha} - \sqrt{\beta}) \partial_{xy} & \sqrt{\alpha} \partial_{yy} + \sqrt{\beta} \partial_{xx} \end{pmatrix}$$

Et donc :

$$\int_{\Omega} \|w\|_d dx dy = \int_{\Omega} \|Q(D)\mathbf{w}\|_d^2$$

Le terme de régularisation devient : $\int \|Q(D)\mathbf{w}\|^2$ et donc puisque nous avons n points, l'intégrale devient une somme $\sum_{i=1}^n \|Q(D)\mathbf{w}(\mathbf{x}_i)\|^2$ car l'échantillonnage est systématique, tandis que le cas aléatoire est réalisé selon une loi uniforme.

La relation 2.5 devient alors :

$$\mathbf{w} = \arg \min \mathbf{w} \in (H^2(\Omega))^2 \sum_{i=1}^n \|\mathbf{w}(\mathbf{x}_i) - \mathbf{w}_i\|^2 + \|Q(D)\mathbf{w}(\mathbf{x}_i)\|^2 \quad (2.6)$$

Ici, les fonctions de base radiales sont utilisées, ce qui correspond simplement au noyau employé dans [1], $\phi(\mathbf{x}) = (128\pi)^{-1} \|\mathbf{x}\|^4 \log \|\mathbf{x}\|$ en supposant que :

$$\mathbf{w}(\mathbf{x}) = \sum_{i=1}^n \phi\left(\frac{\mathbf{x} - \mathbf{x}_i}{h}\right) q_i \quad \text{avec} \quad q_i \in \mathbb{R}^2$$

Avec h un paramètre de lissage. Ce qui nous ramène à minimiser l'expression $J(q)$ suivante :

$$J(q) = \sum_{k=1}^n \sum_{i=1}^n \sum_{j=1}^n \phi(\mathbf{x}_i - \mathbf{x}_k) \phi(\mathbf{x}_j - \mathbf{x}_k) q_i^T (I_2 + Q_i^T Q_j) q_j - 2 \sum_{k=1}^n \sum_{i=1}^n \phi(\mathbf{x}_i - \mathbf{x}_k) q_i^T \mathbf{w}_k + \sum_{k=1}^n \|\mathbf{w}_k\|^2$$

avec $Q_i = Q(D)\mathbf{w}(\mathbf{x}_i)$. En posant :

$$A = \begin{pmatrix} A_{11} & A_{12} & \cdots & A_{1n} \\ A_{21} & A_{22} & \cdots & A_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ A_{n1} & A_{n2} & \cdots & A_{nn} \end{pmatrix}$$

et

$$b_i = \sum_{k=1}^n \phi(\mathbf{x}_i - \mathbf{x}_k) \mathbf{w}_k$$

avec chaque bloc A_{ij} de taille 2×2 .

$$A_{ij} = \sum_{k=1}^n \phi(\mathbf{x}_i - \mathbf{x}_k) \phi(\mathbf{x}_j - \mathbf{x}_k) (I_2 + Q_i^T Q_j)$$

On obtient :

$$J(q) = \sum_{i=1}^n \sum_{j=1}^n q_i^T A_{ij} q_j - 2 \sum_{i=1}^n b_i^T q_i + \|\mathbf{w}_k\|^2$$

Notre problème s'écrit donc sous la forme :

$$J(q) = q^T A q - 2q^T b + c$$

En passant par le calcul du gradient, on obtient $q = (q_1, q_2, \dots, q_n)^T \in \mathbb{R}^{2n}$ qui satisfait la relation suivante :

$$Aq - b = 0$$

Calculer q revient donc à inverser une matrice de taille $2n \times 2n$. Nous avons implémenté cette méthode, mais son fonctionnement n'est pas encore satisfaisant, ce qui explique que ses résultats ne sont pas présentés ici.

Remarque : en complément des méthodes développées sur les observations directes, nous avons aussi exploré et consulté plusieurs articles portant sur l'estimation de champs vectoriels à partir d'images, ainsi que sur l'exploitation des données d'altimétrie pour ce même objectif. Cette approche nous a permis d'enrichir notre compréhension des techniques disponibles pour traiter les observations indirectes. La suite du travail consistera à développer ces méthodes.

CHAPITRE 3

APPLICATION

Dans ce chapitre, nous allons présenter et analyser les résultats obtenus avec les différentes méthodes exposées dans le chapitre précédent. Pour effectuer cette analyse, nous avons besoin de métriques appropriées.

3.1 Métriques

Pour évaluer nos différents travaux, nous utiliserons un certain nombre de métriques et méthodes. Soit $\hat{\mathbf{w}}_i$ le vecteur estimé en un point i et $\mathbf{w}_i$ le vecteur de référence au même point. Nous utilisons deux métriques principales pour évaluer la qualité de nos estimations.

3.1.1 Erreur Quadratique Moyenne (EQM)

$$\text{EQM} = \frac{1}{n} \sum_{i=1}^n \|\mathbf{w}_i - \hat{\mathbf{w}}_i\|^2,$$

où n est le nombre total de points observés.

Interprétation : Cette métrique mesure la différence moyenne en intensité entre les vecteurs estimés et les vecteurs réels. Elle reflète à la fois les erreurs sur la magnitude et la direction. Une valeur faible d'EQM indique que, en moyenne, les vecteurs estimés sont proches des vrais vecteurs en termes de longueur et d'orientation.

3.1.2 Erreur Angulaire Moyenne(EAM)

Pour chaque point i , on calcule l'angle θ_i entre $\mathbf{w}_i$ et $\hat{\mathbf{w}}_i$ par :

$$\theta_i = \arccos \left(\frac{\mathbf{w}_i \cdot \hat{\mathbf{w}}_i}{\|\mathbf{w}_i\| \|\hat{\mathbf{w}}_i\|} \right).$$

L'erreur angulaire moyenne est alors :

$$\frac{1}{n} \sum_{i=1}^n \theta_i.$$

Interprétation : Cette métrique évalue la différence moyenne en orientation entre les vecteurs estimés et les vecteurs vrais, indépendamment de leur norme. Elle est particulièrement utile lorsque la direction du champ vectoriel est plus importante que sa magnitude.

3.2 Protocole expérimental et évaluation

Pour évaluer la sensibilité de nos méthodes au bruit, nous ajoutons artificiellement plusieurs réalisations de bruit à nos données. Pour cela, nous utilisons la loi de von Mises, car pour les champs vectoriels, cette loi est particulièrement adaptée, elle modélise de manière efficace la distribution angulaire des perturbations, ce qui est essentiel pour représenter correctement le bruit directionnel dans ce contexte. On procède de la façon suivante :

- Ramener les composantes u et v du champ en coordonnées polaires, c'est-à-dire, on calcule le rayon et l'angle en chaque point de la grille.
- Générer des observations de loi de Von Mises de moyenne zéro qu'on ajout au angle précédemment calculé.
- Retransformer le champ en coordonnées cartésiennes en utilisant l'angle bruité. On obtient ainsi le champ bruité.

La figure suivante illustre un cas d'application dans lequel un bruit aléatoire de von Mises, de moyenne nulle et de paramètre de concentration $\kappa = 5.0$, a été ajouté.

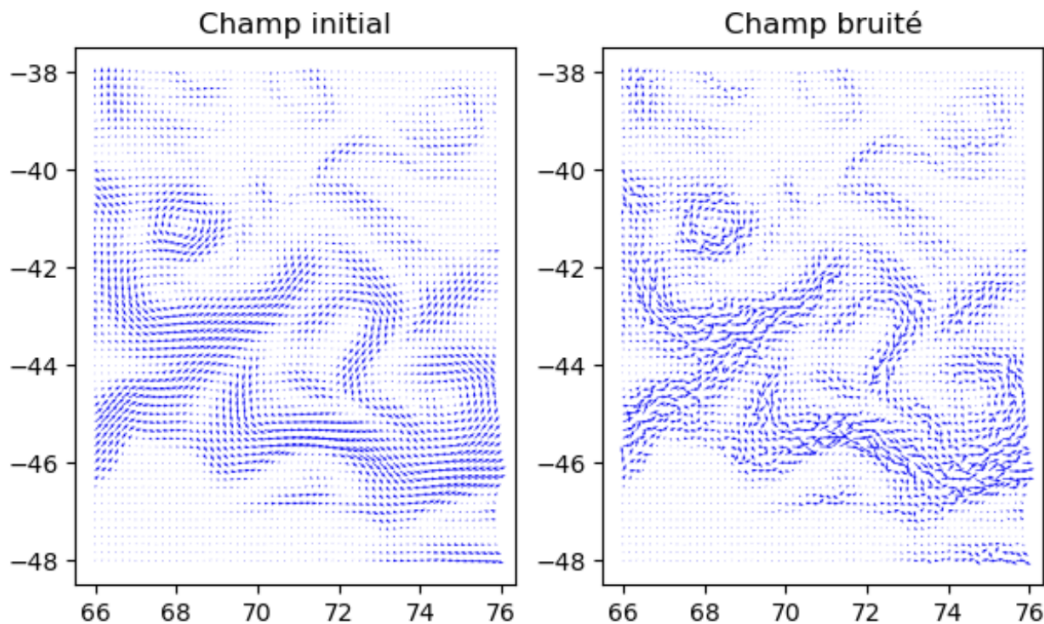


Figure 3.1 – Bruitage avec la loi von Mises.

Il est aussi important de souligner que pour les méthodes ayant des paramètres comme le paramètre de lissage, ces derniers sont optimisés à l'aide d'une validation croisée.

3.3 Résultats

Il est question ici de présenter les différents résultats obtenus et de les analyser.

3.3.1 Analyse des résultats de l'application des méthodes sur les données

Dans un premier temps, les différentes méthodes ont été appliquées sur les champs initial en faisant un sous-échantillon comme décrit dans le chapitre 2, section 2. Les résultats obtenus sont consignés dans les tableaux suivants sachant que nous avons fait un sous-échantillonnage de 50%, 33% et 25%.

Résultats interpolation : Nous présentons ici uniquement les résultats des méthodes d'interpolation.

Méthodes	50%	33%	25%
Int Bilinéaire	2.087	4.333	6.478
Int B-spline	3.278	6.896	9.904
Int Amodei	1.385	3.107	4.999
Int thin plat	1.437	3.322	5.323

Table 3.1 – EAM - Échantillonnage systématique

Méthodes	50%	33%	25%
Int Bilinéaire	4.266	10.802	12.591
Int B-spline	5.604	12.308	19.285
Int Amodei	3.102	7.286	9.012
Int thin plat	3.630	8.106	10.871

Table 3.2 – EAM - Échantillonnage aléatoire

Méthodes	50%	33%	25%
Int Bilinéaire	0.0003	0.0011	0.0023
Int B-spline	0.0004	0.0015	0.0031
Int Amodei	0.0001	0.0005	0.0011
Int thin plat	0.0001	0.0006	0.0014

Table 3.3 – EQM - Échantillonnage Systématique

Méthodes	50%	33%	25%
Int Bilinéaire	0.0016	0.0077	0.0101
Int B-spline	0.0017	0.0081	0.0172
Int Amodei	0.0006	0.0025	0.0042
Int thin plat	0.0010	0.0037	0.0067

Table 3.4 – EQM - Échantillonnage aléatoire

L'analyse des résultats présentés dans les tableaux 3.1 à 3.4 montre que l'augmentation du taux d'échantillonnage améliore significativement la précision des reconstructions, quel que soit le type d'échantillonnage ou la méthode utilisée. En effet, l'EAM et l'EQM croissent de manière marquée lorsque le taux d'échantillonnage passe de 50 % à 25%, illustrant l'impact négatif de la perte d'information, ce qui était attendu. Les performances des échantillonnages systématique et aléatoire sont globalement comparables, bien que le systématique présente souvent un léger avantage en précision. Parmi les méthodes testées, la méthode d'Amodei et celle basée sur les splines plaque mince offrent les meilleures performances. Les B-splines donnent des résultats intermédiaires mais restent sensibles à la diminution du taux d'échantillonnage. Dans l'ensemble, ces observations confirment que le maintien d'un taux d'échantillonnage élevé et le choix judicieux de la méthode d'interpolation sont déterminants pour limiter les erreurs et améliorer la qualité des estimations.

Résultats interpolation : Il est présenté ici les résultats des méthodes d'interpolation et d'approximation en présence de bruit.

Méthodes	50%	33%	25%
Int Bilinéaire	20.024	24.225	25.820
Int B-spline	31.404	32.35	35.764
Int Amodéi	23.096	26.358	27.896
App thin plat	16.379	19.096	22.436

Table 3.5 – EAM - Échantillonnage systématique + bruit

Méthodes	50%	33%	25%
Int Bilinéaire	19.146	23.436	24.461
Int B-spline	31.305	29.132	38.378
Int Amodéi	21.158	25.518	26.606
App thin plat	23.453	26.715	28.465

Table 3.6 – EAM - Échantillonnage aléatoire + bruit

Méthodes	50%	33%	25%
Int Bilinéaire	0.0219	0.0247	0.0266
Int B-spline	8.64×10^9	1.7505	0.1557
Int Amodéi	0.0300	0.0288	0.0339
App thin plat	0.0134	0.0162	0.0236

Table 3.7 – EQM - Échantillonnage Systématique + bruit

Méthodes	50%	33%	25%
Int Bilinéaire	0.0199	0.0229	0.0280
Int B-spline	1.81×10^8	0.1087	0.7534
Int Amodéi	0.0237	0.0276	0.0307
App thin plat	0.0271	0.0360	0.0352

Table 3.8 – EQM - Échantillonnage aléatoire + bruit

Légende :**Int** = Interpolation**App** = Approximation

Les résultats montrent qu'en présence de bruit, la méthode d'Amodéi offre les meilleures performances globales en termes d'erreur angulaire moyenne (EAM), avec des valeurs systématiquement plus faibles que celles des autres méthodes, aussi bien en échantillonnage systématique qu'aléatoire. L'interpolation bilinéaire reste compétitive, mais se dégrade rapidement lorsque le taux d'échantillonnage diminue ; de plus, ses reconstructions visuelles apparaissent moins satisfaisantes que celles obtenues par la méthode d'Amodéi ou les splines plaque mince. L'interpolation B-spline présente des erreurs plus élevées et surtout instables, comme le montrent les valeurs extrêmes de l'EQM. L'approximation par spline plaque mince se révèle plus robuste au bruit, avec des EQM faibles et stables, mais au prix d'EAM supérieurs à ceux de la méthode d'Amodéi, ce qui s'explique par le fait que cette dernière intègre les propriétés physiques via la norme div-curl. Par ailleurs, l'échantillonnage aléatoire et systématique produisent des performances comparables, bien que l'aléatoire entraîne parfois une dispersion un peu plus marquée. Globalement, la méthode d'Amodéi apparaît comme la plus efficace pour concilier fidélité et robustesse, tandis que l'approximation par spline plaque mince fournit néanmoins des résultats intéressants.

3.3.2 Analyse de la sensibilité au bruit

L'objectif ici est d'évaluer la sensibilité au bruit des méthodes particulièrement les méthodes d'approximation pour cela, le processus de bruitage décrit précédemment est appliqué une centaine de reprise aux données d'entrée. Pour chaque réalisation, les erreurs ou métriques de performance sont calculées. L'ensemble des erreurs obtenues est ensuite analysé à l'aide de boîtes à moustaches, afin de comparer la robustesse des méthodes face au bruit. En complément, nous utilisons des diagrammes en violon pour visualiser la forme des distributions d'erreurs.

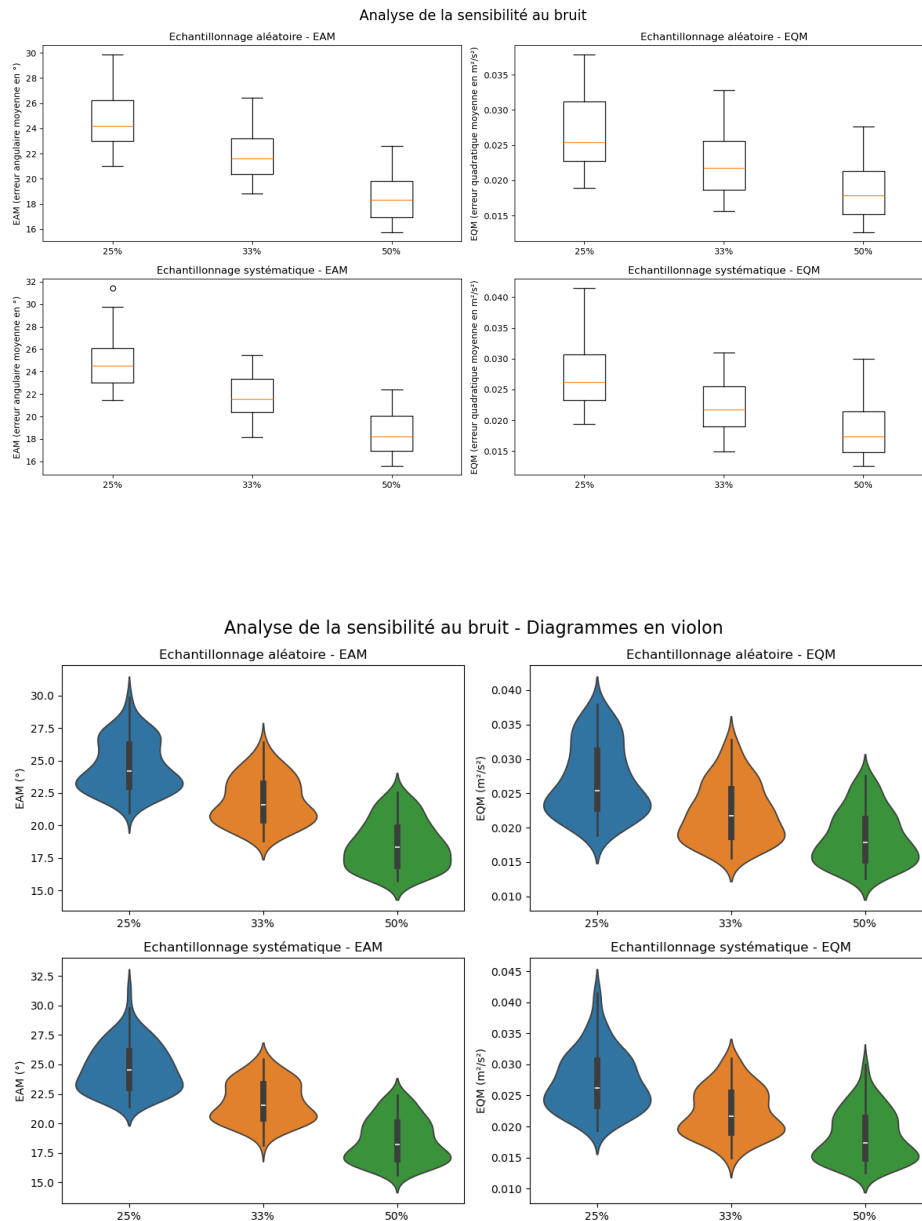


Figure 3.2 – Comparaison des erreurs sous variations de bruit pour échantillonnage aléatoire et systématique

Les figures précédentes présentent les boxplots et violin plots des erreurs sur le champ bruiteur selon la loi de von Mises, obtenus pour 100 bruits différents. Ces représentations illustrent la dispersion et la distribution des erreurs sous différentes conditions de bruit, permettant d'évaluer la robustesse des méthodes d'échantillonnage face aux perturbations. On observe ainsi que, malgré les variations du bruit, l'augmentation du taux d'échantillonnage de 25 % à 50% améliore notablement les performances, en réduisant significativement les erreurs angulaires moyennes (EAM) et quadratiques moyennes (EQM) pour les deux méthodes d'échantillonnage. Cette amélioration se traduit également par des distributions d'erreurs plus concentrées et moins dispersées, témoignant d'une plus grande stabilité et fiabilité des résultats obtenus. Par ailleurs, les performances des échantillonnages aléatoires et systématiques restent globalement comparables.

Comparons à présent la sensibilité au bruit de la méthode d'Amodei et les splines plaque mince.

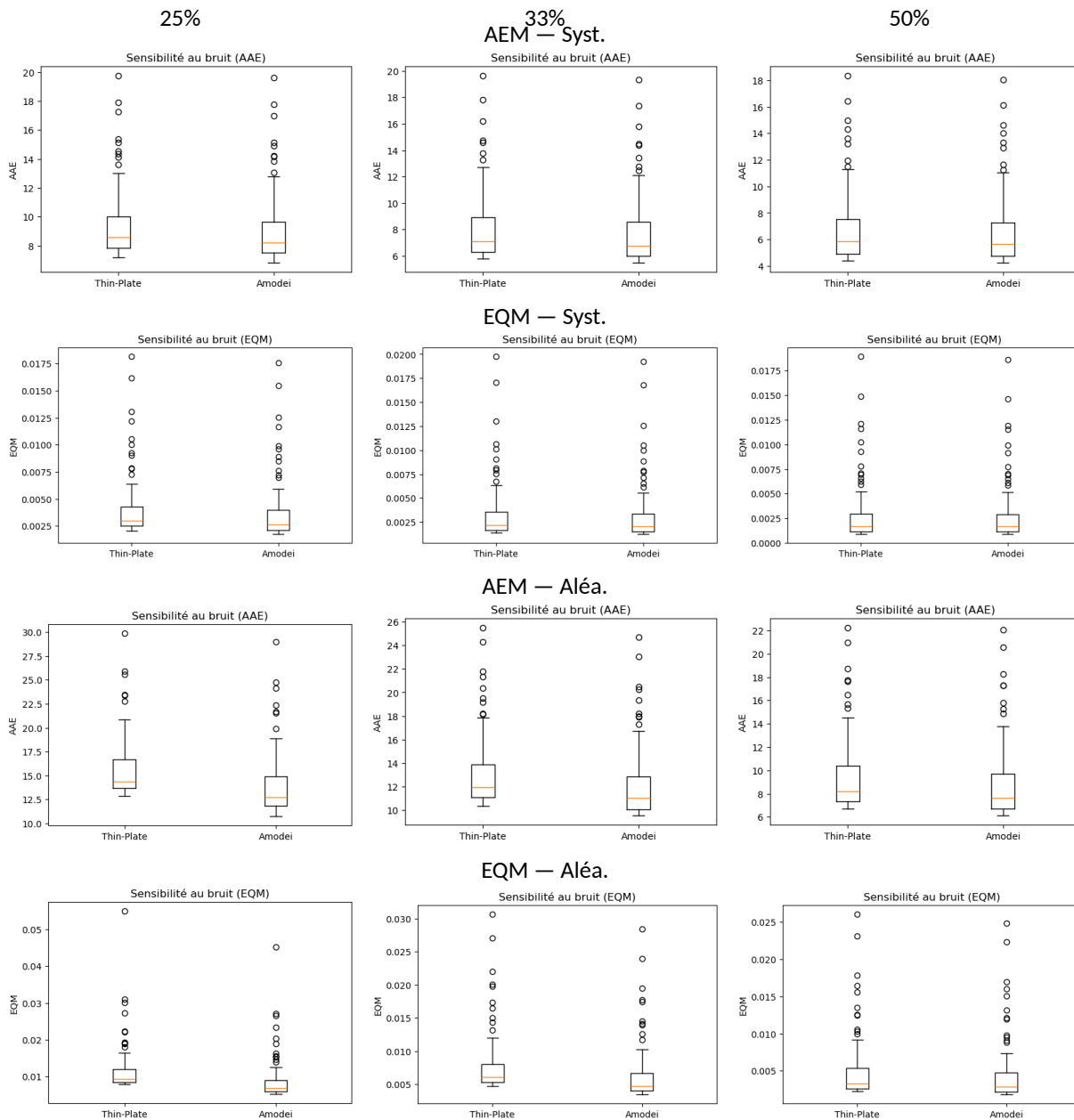


Figure 3.3 – Comparaison AEM/EQM pour échantillonnages systématique et aléatoire.

Les boîtes à moustaches permettent d'analyser la sensibilité au bruit des deux méthodes (Thin Plate et Amodéi) pour différents taux d'échantillonnage et types de schémas. On constate que la méthode d'Amodéi présente systématiquement des erreurs angulaires moyennes (AEM) plus faibles que la méthode Thin Plate, avec des distributions plus resserrées et moins de valeurs extrêmes. Cela traduit une meilleure stabilité et une robustesse accrue face au bruit. En revanche, la méthode Thin Plate, bien qu'elle puisse parfois approcher des performances similaires en moyenne, montre une plus grande dispersion des erreurs et un nombre plus important de valeurs aberrantes, ce qui la rend plus sensible au bruit. Concernant l'erreur quadratique moyenne (EQM), les deux méthodes conservent des valeurs faibles, mais la méthode d'Amodéi obtient également des distributions plus concentrées. Enfin, l'échantillonnage aléatoire dégrade globalement les performances par rapport à la systématique, en particulier pour la méthode Thin Plate, tandis qu'Amodéi demeure plus stable dans les deux cas.

3.4 Discussion

Bien que les méthodes proposées dans ce stage aient démontré de bonnes performances, elles ne sont pas entièrement adaptées au problème traité, car elles n'exploitent pas les relations physiques reliant les composantes du champ comme la méthode d'Amodéi, relations qui peuvent influencer les estimations réalisées. Néanmoins, on constate une performance notable de l'interpolation et l'approximation spline plaque mince, aussi bien sur le plan visuel que selon les indicateurs quantitatifs tels que l'erreur quadratique moyenne et l'erreur angulaire moyenne. La méthode proposée par Amodéi, en revanche, intègre les propriétés physiques du champ via la norme div-curl, ce qui lui confère un avantage en précision. La méthode d'Amodéi s'est révélée la plus performante en présence de bruit, grâce à l'intégration des contraintes physiques via la norme div-curl, ce qui lui confère robustesse et stabilité. Les splines plaque mince régularisées donnent des résultats intéressants, mais restent sensibles au bruit et nécessitent une bonne répartition des points pour être fiables. En comparaison, l'interpolation bilinéaire donne également des résultats acceptables en présence du bruit mais qui sont pas trop bon visuellement.

Tout le travail a été effectué en Python. La majorité des méthodes ont été codées manuellement, et, lorsque des packages ont été utilisés, nos propres versions avaient d'abord été testées.

3.5 Difficultés rencontrées

Au cours de ce stage, plusieurs difficultés ont été rencontrées, liées à la nature technique et interdisciplinaire du sujet. En effet, l'étude menée nécessitait la maîtrise de notions variées, allant de l'analyse des splines à des concepts avancés en équations aux dérivées partielles (EDP). N'ayant pas au départ l'ensemble des prérequis théoriques, il a été indispensable de consacrer un temps considérable à l'auto-formation, en consultant des supports pédagogiques, des articles scientifiques et en réalisant des exercices pratiques pour mieux comprendre et manipuler ces outils.

Une autre difficulté majeure résidait dans le manque de ressources bibliographiques spécifiques au thème exact du stage. Si certaines notions générales sur les splines ou les EDP sont bien documentées, les travaux portant sur leur application combinée dans le contexte précis de notre problématique se sont avérés rares. Cela a parfois rendu la recherche d'informations pertinentes laborieuse, nécessitant de croiser des sources issues de domaines différents afin de reconstituer un cadre théorique cohérent.

Enfin, la mise en œuvre pratique des méthodes étudiées, notamment pour la reconstruction de champs à partir de données bruitées, a exigé de nombreuses expérimentations et ajustements. L'adaptation des algorithmes aux contraintes spécifiques du problème, le choix des paramètres de régularisation et la prise en compte des effets du bruit ont représenté des étapes délicates, mais également formatrices.

3.6 Suggestions

À l'issue de ce stage, plusieurs pistes d'amélioration et de prolongement pour ma future thèse peuvent être envisagées.

La première étape consistera à finaliser et à approfondir les méthodes avec contraintes physiques proposées ici. Ces contraintes, intégrant des connaissances a priori sur le phénomène étudié, sont essentielles pour renforcer la robustesse et la précision des modèles développés. Leur mise au point constituera une base solide pour les travaux

futurs.

Ensuite, il conviendra d'étendre ces approches aux observations indirectes issues de données d'imagerie, notamment les mesures d'altimétrie. L'exploitation de ces sources complémentaires permettra d'enrichir la modélisation et d'améliorer la caractérisation du phénomène, particulièrement dans des situations où les données directes sont limitées ou difficiles d'accès.

Enfin, il sera crucial d'explorer et d'intégrer d'autres métriques d'évaluation, adaptées aux spécificités des données et des modèles employés. Une évaluation plus diversifiée et rigoureuse favorisera une meilleure compréhension des performances et des limites des méthodes développées, orientant ainsi leur optimisation.

Ces axes constitueront les fondements de ma future thèse, qui visera à approfondir ces aspects méthodologiques tout en explorant de nouvelles pistes applicatives.

CONCLUSION

L'estimation d'un champ vectoriel à partir de données incomplètes, bruitées ou indirectes est un problème mal posé, car le champ estimé peut varier considérablement même pour de petites perturbations des données, en particulier en présence de bruit. De plus, plusieurs champs peuvent parfois expliquer les mêmes observations partielles. Autrement dit, l'estimation idéale n'est généralement pas connue. Les approches classiques simples, telles que l'interpolation bilinéaire, les B-splines et les splines plaque mince, ont montré des performances satisfaisantes, notamment la spline plaque mince qui présente une bonne qualité visuelle et des indicateurs quantitatifs encourageants. Cependant, ces méthodes ne tiennent pas pleinement compte des relations physiques intrinsèques au champ étudié, ce qui limite leur précision. La méthode de L. Amodéi, intégrant la norme div-curl pour prendre en compte ces contraintes physiques, offre une meilleure robustesse et une amélioration notable des estimations, bien que ses performances restent proches de celles des splines plaque mince.

Il est également important d'étudier des méthodes d'approximation capables de mieux gérer le bruit, car en sa présence, une interpolation stricte peut entraîner une amplification des erreurs ou des estimations instables. Les splines plaque mince régularisées ont montré des performances correctes, même si elles restent sensibles au bruit et à la répartition des données. En revanche, la méthode d'Amodéi a confirmé son efficacité, notamment grâce à l'utilisation de la norme div-curl, qui permet d'intégrer directement des contraintes physiques dans la régularisation. Cela constitue une piste prometteuse pour la méthode d'approximation que nous développons actuellement, où l'on cherche à mieux prendre en compte les propriétés physiques du champ.

En définitive, ces travaux ouvrent des perspectives prometteuses pour l'intégration de contraintes physiques plus poussées et pour l'exploitation de données d'observations indirectes, telles que les mesures d'altimétrie. La poursuite de ces recherches, notamment dans le cadre d'une thèse, visera à améliorer la robustesse et la précision des modèles en tenant compte des caractéristiques physiques du champ et en développant des méthodologies adaptées aux données bruitées et partiellement échantillonnées. Ces méthodes pourraient également être testées sur différents types de données, tels que les champs de vent.

BIBLIOGRAPHIE

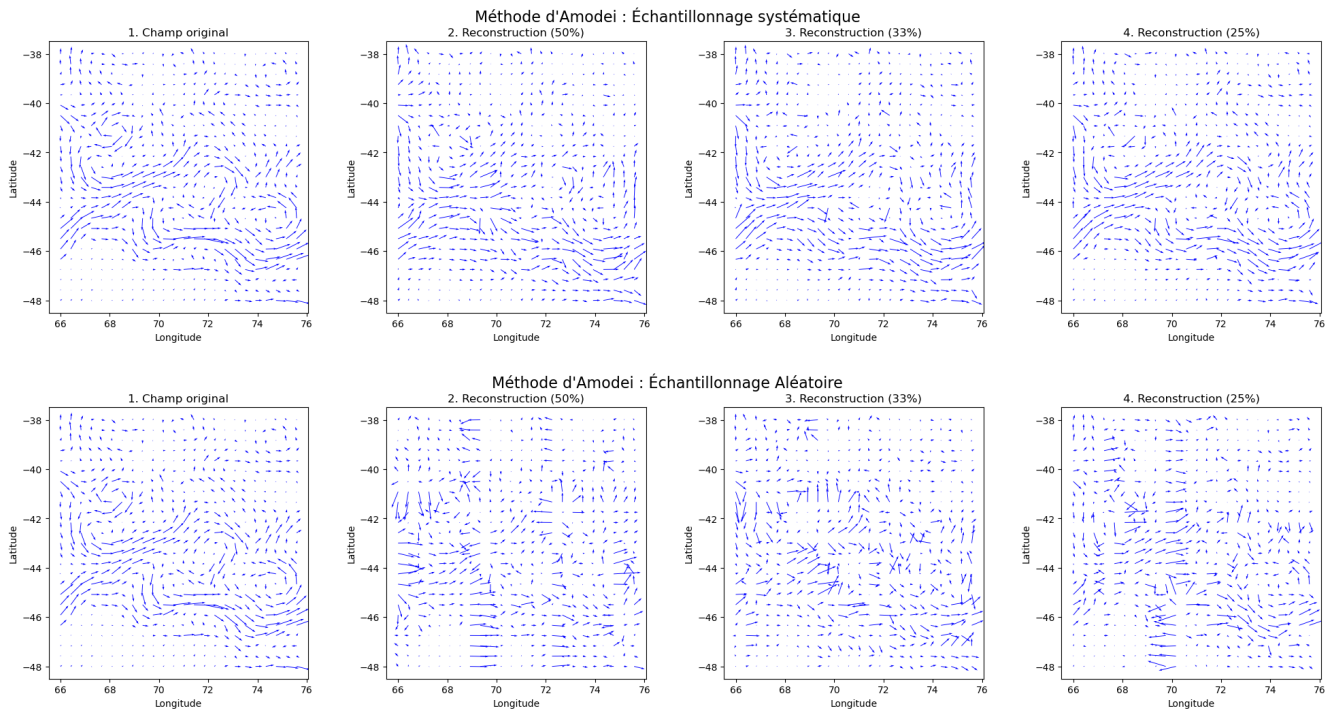
- [1] Amodei, L. et Benbourhim, M. N. "A vector spline approximation". In : *Journal of Approximation Theory* 67.1 (1991), p. 51-79. doi : 10.1016/0021-9045(91)90025-6. url : <https://www.sciencedirect.com/science/article/pii/0021904591900256>.
- [2] Arfken, George B. et Weber, Hans J. *Mathematical Methods for Physicists*. 7th. Academic Press, 2013, p. 143-152.
- [3] Boor, Carl de. *A Practical Guide to Splines*. Sous la dir. de J.E. Marsden et L. Sirovich. Revised Edition. T. 27. Applied Mathematical Sciences. New York : Springer, 2001. isbn : 978-0-387-95366-3. doi : 10.1007/978-1-4612-6333-3.
- [4] Brezis, Haim. *Analyse fonctionnelle : Théorie et applications*. Sous la dir. de P. G. Ciarlet et J. L. Lions. 2e tirage. Mathématiques appliquées pour la maîtrise. Paris : Masson, 1983.
- [5] Copernicus Marine Service. *Copernicus Marine Service*. Consulté le 23 juin 2025. 2025. url : <https://marine.copernicus.eu/fr/explainers/why-ocean-important/ocean-currents>.
- [6] Curry, H. B. et Schoenberg, I. J. "On Polya Frequency Functions, IV : The Fundamental Spline Functions and their Limits". In : *Journal d'Analyse Mathématique* 17.1 (1966), p. 71-107.
- [7] *Fonction de base radiale*. Wikipédia, l'encyclopédie libre. Consulté le 10 avril 2025. url : https://fr.wikipedia.org/wiki/Fonction_de_base_radiale.
- [8] Guinet, Christophe et al. "Big data analyses reveal patterns and drivers of the movements of southern elephant seals". In : *Scientific Reports* 8.1 (2018), p. 1-10. doi : 10.1038/s41598-019-44970-1. url : <https://www.nature.com/articles/s41598-019-44970-1>.
- [9] Schumaker, L. L. "On Shape Preserving Quadratic Spline Interpolation". In : *SIAM Journal on Numerical Analysis* 20.4 (1983), p. 854-864.
- [10] Till Isambert, Jean-Paul Berroir et Herlin, Isabelle. "A Multi-scale Vector Spline Method for Estimating the Fluids Motion on Satellite Images". In : *European Conference on Computer Vision - ECCV 2008*. Inria ID : inria-00583017. Marseille, France : Springer, 2008, p. 665-676. doi : 10.1007/978-3-540-88693-8_49. url : https://doi.org/10.1007/978-3-540-88693-8_49.

- [11] Waterford Maritime. *Understanding Ocean Currents : Their Role in Climate, Navigation, and Marine Life*. Consulté le 14 août 2025. 2024. url : <https://waterfordmaritime.org/understanding-ocean-currents-their-role-in-climate-navigation-and-marine-life/>.
- [12] Wood, Simon N. *Generalized Additive Models : An Introduction with R*. 2nd. Chapman et Hall/CRC, 2017, p. 215-216.

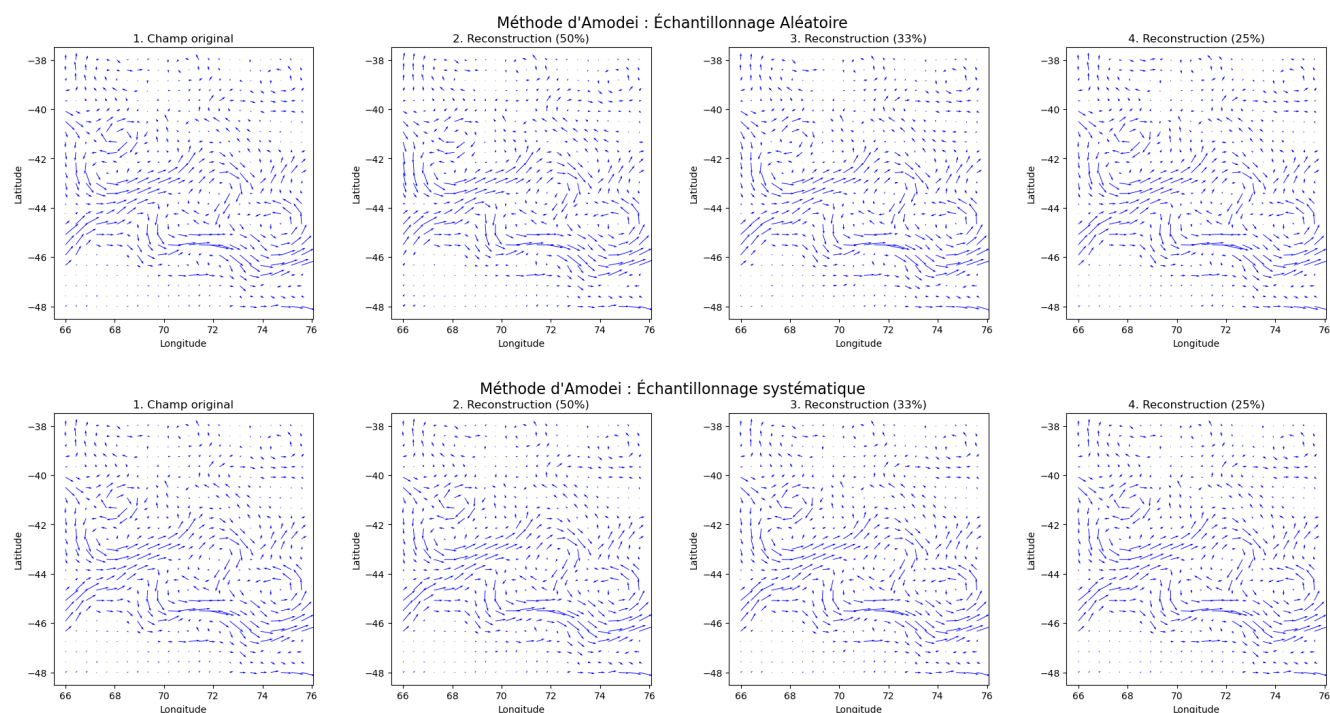
ANNEXE

Méthode d'Amodéi - Complément d'images

Pour : $\alpha = 0.01$ et $\beta = 0.99$



Pour : $\alpha = 0.05$ et $\beta = 0.5$



Interpolation en présence du bruit - Complément d'images

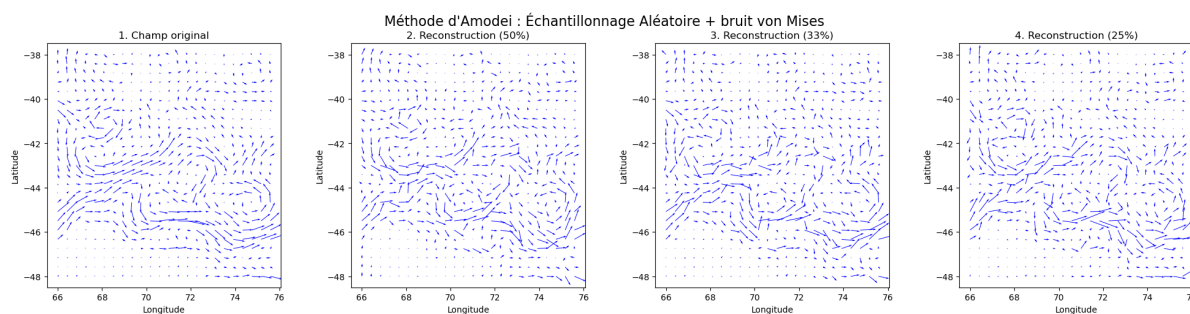


Figure 3.4 – Méthode d'Amodéi — Échantillonnage aléatoire

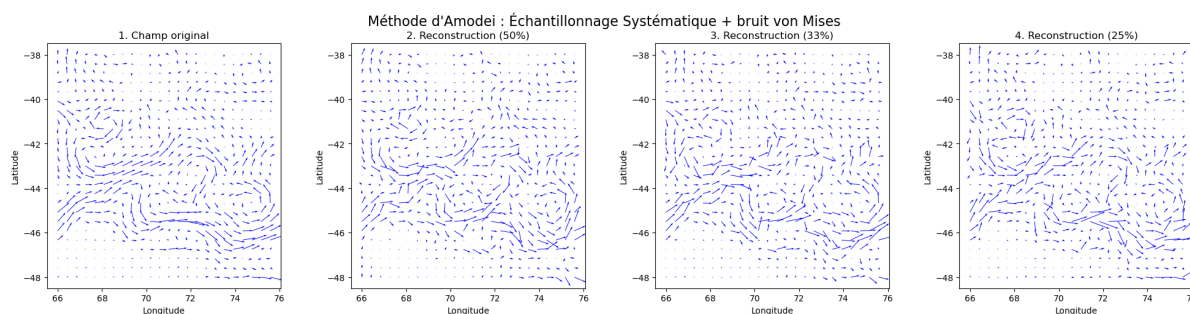


Figure 3.5 – Méthode d'Amodéi — Échantillonnage systématique

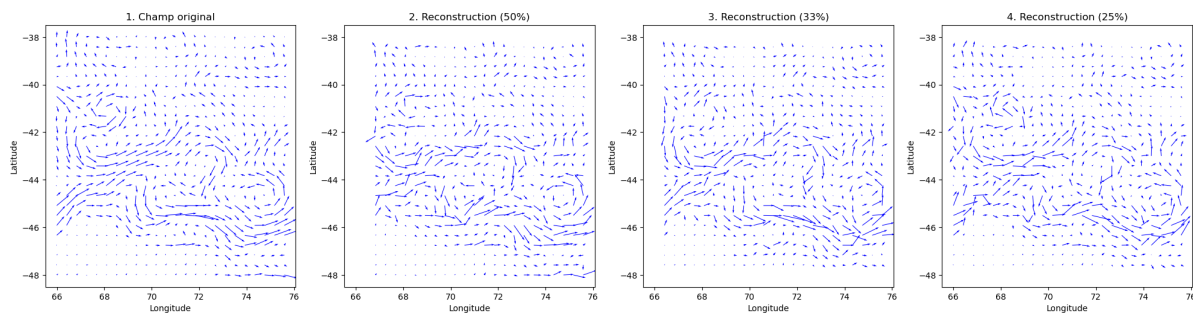


Figure 3.6 – Interpolation bilinéaire — Échantillonnage aléatoire

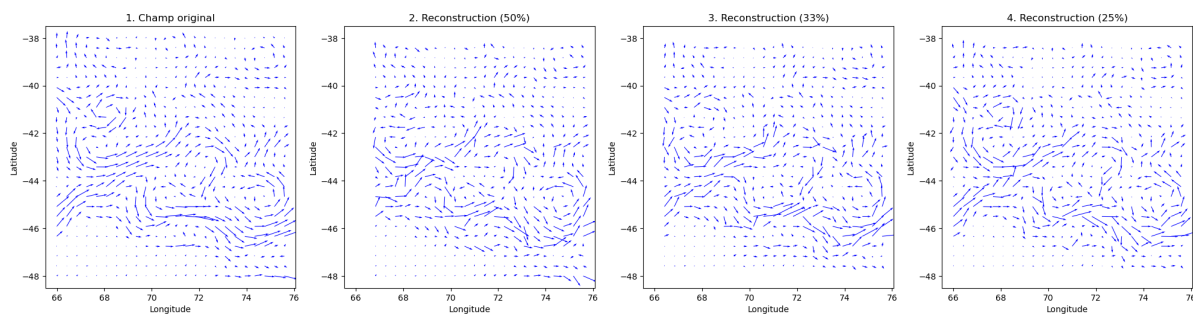


Figure 3.7 – Interpolation bilinéaire — Échantillonnage systématique

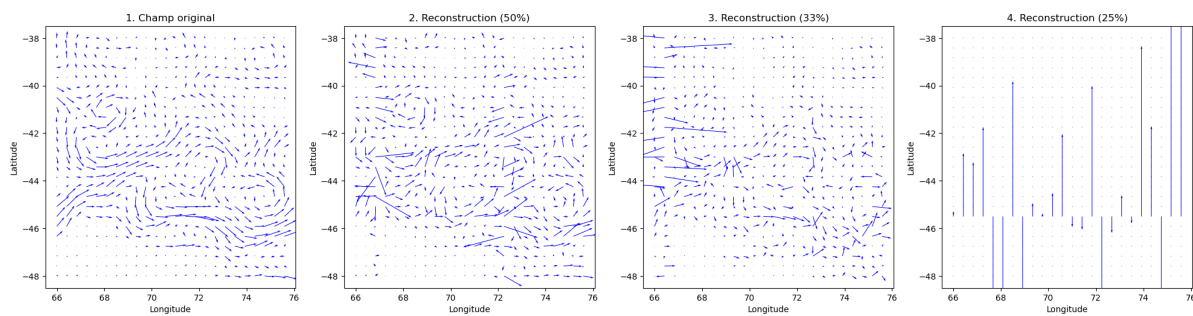


Figure 3.8 – Interpolation B-spline

Code Python

```

1
2 import xarray as xr
3 import numpy as np
4 import pandas as pd
5 import matplotlib.pyplot as plt
6 import matplotlib.colors as colors
7 from matplotlib.animation import FuncAnimation
8 from scipy.interpolate import griddata, SmoothBivariateSpline, Rbf
9
10 ALPHA = .9

```

```

11 BETA = .1
12 EPS = 1e-6
13 TOL = 1e-12
14
15 titles = [
16     "1. Champ original",
17     "2. Reconstruction (50%)",
18     "3. Reconstruction (33%)",
19     "4. Reconstruction (25%)"
20 ]
21 noms_champs = ["Reconstruction 25\\%", "Reconstruction 33\\%", "Reconstruction 50\\%"]
22
23 def aae(u, v, u_hat, v_hat):
24     """Erreur angulaire moyenne entre deux champs vectoriels
25     (u,v) champ initial
26     (u_hat,v_hat) champ estimé """
27     dot_product = u * u_hat + v * v_hat
28     norm_true = np.sqrt(u**2 + v**2)
29     norm_est = np.sqrt(u_hat**2 + v_hat**2)
30     cos_angle = np.clip(dot_product / (norm_true * norm_est + 1e-8), -1.0, 1.0)
31     angle = np.degrees(np.arccos(cos_angle))
32     return np.mean(angle)
33
34 def eqm(u, v, u_hat, v_hat):
35     """(u,v) champ initial
36     (u_hat,v_hat) champ estimé
37     """
38     return np.mean((u - u_hat)**2 + (v - v_hat)**2)
39
40
41 def echantillon_aleatoire(u, v, percent=0.50):
42     nlat, nlon = u.shape
43
44     i_lat = np.random.choice(u['latitude'], size=int(nlat*percent), replace=False)
45     i_lon = np.random.choice(u['longitude'], size=int(nlon*percent), replace=False)
46
47     u_sample = u.sel(latitude=i_lat, longitude=i_lon)
48     v_sample = v.sel(latitude=i_lat, longitude=i_lon)
49     return u_sample, v_sample
50
51 def echantillon_systematique(u, v, percent=0.50):
52     step = int(1/percent)

```

```

53     u_sample = u[:,::step, ::step]
54     v_sample = v[:,::step, ::step]
55     return u_sample, v_sample
56
57 def noise_von_mises(u, v, kappa=5.0):
58     """Ajoute du bruit de Von Mises à un champ vectoriel"""
59     angles = np.arctan2(v, u)
60     normes = np.sqrt(u**2 + v**2)
61     angles_noise = angles + np.random.vonmises(mu=np.zeros(u.shape), kappa=kappa)
62     noise_u = normes * np.cos(angles_noise)
63     noise_v = normes * np.sin(angles_noise)
64     return noise_u, noise_v
65
66
67
68
69 def interpolate_vector_field_from_xarray(u, v, lon_target, lat_target,
↪ method='linear', s=0):
70     """
71     Interpole un champ vectoriel (u, v) donné sous forme de xarray.DataArray.
72
73     Paramètres :
74     - u, v : xarray.DataArray avec dimensions contenant 'lon' et 'lat'
75     - lon_target, lat_target : grilles cibles (np.ndarray)
76     - method : 'linear', 'spline', 'thin_plate'
77     - s : paramètre de lissage (pour spline et thin_plate)
78
79     Retourne :
80     - u_interp, v_interp : champs interpolés (np.ndarray de même forme que
↪ lon_target)
81     """
82     # Extraction des coordonnées
83     lon = u['longitude'].values
84     lat = u['latitude'].values
85     lon_grid, lat_grid = np.meshgrid(lon, lat)
86
87     # Aplatir les grilles et valeurs
88     points = np.column_stack((lon_grid.ravel(), lat_grid.ravel()))
89     u_values = u.values.ravel()
90     v_values = v.values.ravel()
91
92     target_points = np.column_stack((lon_target.ravel(), lat_target.ravel()))

```



```

93
94     if method == 'linear':
95         u_interp = griddata(points, u_values, target_points, method='linear')
96         v_interp = griddata(points, v_values, target_points, method='linear')
97
98     elif method == 'spline':
99         spline_u = SmoothBivariateSpline(lon_grid.ravel(), lat_grid.ravel(),
100             ↪ u_values, s=s)
101         spline_v = SmoothBivariateSpline(lon_grid.ravel(), lat_grid.ravel(),
102             ↪ v_values, s=s)
103         u_interp = spline_u.ev(lon_target.ravel(), lat_target.ravel())
104         v_interp = spline_v.ev(lon_target.ravel(), lat_target.ravel())
105
106     elif method == 'thin_plate':
107         rbf_u = Rbf(lon_grid.ravel(), lat_grid.ravel(), u_values,
108             ↪ function='thin_plate', smooth=s)
109         rbf_v = Rbf(lon_grid.ravel(), lat_grid.ravel(), v_values,
110             ↪ function='thin_plate', smooth=s)
111         u_interp = rbf_u(lon_target.ravel(), lat_target.ravel())
112         v_interp = rbf_v(lon_target.ravel(), lat_target.ravel())
113     else:
114         raise ValueError("Méthode inconnue. Utilisez 'linear', 'spline' ou
115             ↪ 'thin_plate'.")
116
117     shape = lon_target.shape
118     return u_interp.reshape(shape), v_interp.reshape(shape)
119
120 def plot_vector_fields_comparison(lon, lat, vector_fields, titles=titles, skip=5,
121     ↪  subtitle=None, color='blue'):
122     """
123     Affiche une série de champs vectoriels côte à côte.
124
125     Paramètres :
126     - lon, lat : grilles (2D) de coordonnées pour tous les champs
127     - vector_fields : liste de tuples [(u1, v1), (u2, v2), ...]
128     - titles : liste de titres pour chaque subplot
129     - subtitle : titre général (en haut)
130     - color : couleur des flèches (par défaut : 'blue')
131     """
132     n = len(vector_fields)
133     fig, axes = plt.subplots(1, n, figsize=(5 * n, 5))

```

```

129
130     if n == 1:
131         axes = [axes]
132
133     for i, (u, v) in enumerate(vector_fields):
134
135         lon_skip = lon[:, ::skip, ::skip]
136         lat_skip = lat[:, ::skip, ::skip]
137         u_skip = u[:, ::skip, ::skip]
138         v_skip = v[:, ::skip, ::skip]
139
140         axes[i].quiver(lon_skip, lat_skip, u_skip, v_skip, color=color)
141         axes[i].set_xlabel("Longitude")
142         axes[i].set_ylabel("Latitude")
143         axes[i].set_aspect('equal')
144
145         if titles and i < len(titles):
146             axes[i].set_title(titles[i])
147
148     if supitle:
149         plt.suptitle(supitle, fontsize=16)
150     plt.tight_layout()
151     plt.show()
152
153
154 def evaluer_champs(vector_fields, noms):
155     """
156     Évalue plusieurs champs de vecteurs par rapport à une référence.
157
158     Parameters:
159         vector_fields : list of (u, v)
160             Liste de champs, le premier étant la référence.
161         noms : list of str
162             Noms des champs, même ordre que vector_fields.
163         AAE : function
164             Fonction de calcul de l'Erreur Absolue Moyenne.
165         EQM : function
166             Fonction de calcul de l'Erreur Quadratique Moyenne.
167
168     Returns:
169         DataFrame avec AAE, EQM et L1.
170     """

```

```

171     u_ref, v_ref = vector_fields[0]
172     resultats = []
173
174     for nom, (u, v) in zip(noms, vector_fields[1:]):
175         resultats.append({
176             "Champ": nom,
177             "AAE": float(np.round(aae(u, v, u_ref, v_ref),3)),
178             "EQM": float(np.round(eqm(u, v, u_ref, v_ref),4))
179             "Norme L2": norme_L2(u, v, u_ref, v_ref)
180         })
181
182     return pd.DataFrame(resultats)
183
184
185     import sympy as sp
186     from sympy import symbols, diff
187     alpha=1.0
188     beta=1.0
189
190
191     eps = sp.Symbol('eps', positive=True)
192     x, y = symbols('x y', real=True)
193     r2 = x**2 + y**2
194
195     cK = -sp.Rational(1, 2) / sp.pi
196     K = cK * (r2**2) * sp.log(sp.sqrt(r2) + eps)
197
198     Kxx = diff(K, x, x)
199     Kyy = diff(K, y, y)
200     Kxy = diff(K, x, y)
201
202     fx = sp.lambdify((x, y, eps), Kxx, 'numpy')
203     fy = sp.lambdify((x, y, eps), Kyy, 'numpy')
204     fxy = sp.lambdify((x, y, eps), Kxy, 'numpy')
205
206     # Noyau
207     def kd(r, s, epsilon=1e-9):
208         u, v = r[0] - s[0], r[1] - s[1]
209         xx = float(fx(u, v, epsilon))
210         yy = float(fy(u, v, epsilon))
211         xy = float(fxy(u, v, epsilon))
212         M = np.array([[xx, xy], [xy, yy]], dtype=float)

```

```

213     M[~np.isfinite(M)] = 0.0
214     return M
215
216 def kr(r, s, epsilon=1e-9):
217     u, v = r[0] - s[0], r[1] - s[1]
218     xx = float(fx(u, v, epsilon))
219     yy = float(fy(u, v, epsilon))
220     xy = float(fxy(u, v, epsilon))
221     M = np.array([[yy, -xy], [-xy, xx]], dtype=float)
222     M[~np.isfinite(M)] = 0.0
223     return M
224
225 def KD_KR(X, epsilon=1e-9):
226     N = X.shape[0]
227     Kd = np.zeros((2*N, 2*N), dtype=float)
228     Kr = np.zeros((2*N, 2*N), dtype=float)
229     for i in range(N):
230         for j in range(N):
231             if i == j:
232                 # diagonale nulle
233                 Kd[2*i:2*i+2, 2*j:2*j+2] = 0.0
234                 Kr[2*i:2*i+2, 2*j:2*j+2] = 0.0
235             else:
236                 Kd[2*i:2*i+2, 2*j:2*j+2] = kd(X[i], X[j], epsilon)
237                 Kr[2*i:2*i+2, 2*j:2*j+2] = kr(X[i], X[j], epsilon)
238     # symétrisation numérique
239     Kd = 0.5*(Kd + Kd.T)
240     Kr = 0.5*(Kr + Kr.T)
241     return Kd, Kr
242
243 # Coefficients de la base polynomiale
244 def params(X, u, v, alpha=0.5, beta=0.5, epsilon=1e-9):
245     """
246     Résout  $[(1/\alpha)Kd + (1/\beta)Kr, P], [P^T, 0]$   $[a, b, c, d]^T = [u, v, 0]^T$ 
247      $P = \text{kron}(I2, [1, x, y]) \rightarrow$  base polynomiale de degré 1.
248     """
249     X = np.asarray(X, float); u = np.asarray(u, float); v = np.asarray(v, float)
250     N = X.shape[0]; assert X.shape[1] == 2 and u.size == N and v.size == N
251
252     Kd, Kr = KD_KR(X, epsilon)
253     O = np.column_stack([np.ones(N), X[:,0], X[:,1]]) # (N,3)
254     P = np.kron(np.eye(2), O) # (2N,6)

```

```

255     uv = np.concatenate((u, v))                                # (2N,)
256
257     K = (1/alpha)*Kd + (1/beta)*Kr
258     A = np.block([[K, P],
259                  [P.T, np.zeros((6, 6))]])
260     B = np.concatenate([uv, np.zeros(6)])
261
262
263     try:
264         sol = np.linalg.solve(A, B)
265     except np.linalg.LinAlgError:
266         sol = np.linalg.lstsq(A, B, rcond=None)[0]
267
268     a = sol[0:N]
269     b = sol[N:2*N]
270     c_vec = sol[2*N:2*N+3]          # (c0, c1, c2)
271     d_vec = sol[2*N+3:2*N+6]        # (d0, d1, d2)
272     return a, b, c_vec, d_vec
273
274 def vitesse(x1, y1, X, a, b, c, d,
275            alpha=1.0, beta=1.0, epsilon=1e-6,
276            tol=1e-12, normalize=None):
277
278     X = np.asarray(X, float)
279
280
281     Zx = x1 - X[:, 0]
282     Zy = y1 - X[:, 1]
283     r2 = Zx*Zx + Zy*Zy
284     mask0 = r2 <= tol
285
286     with np.errstate(divide='ignore', invalid='ignore'):
287         dxx = fx(Zx, Zy, epsilon)
288         dyy = fy(Zx, Zy, epsilon)
289         dxy = fxy(Zx, Zy, epsilon)
290
291     dxx = np.nan_to_num(dxx); dyy = np.nan_to_num(dyy); dxy = np.nan_to_num(dxy)
292     dxx[mask0] = 0.0; dyy[mask0] = 0.0; dxy[mask0] = 0.0
293
294     Oq = np.array([1.0, x1, y1, x1*x1, x1*y1, y1*y1], float)
295
296     u = (c @ Oq

```

```

297         + (1/alpha)*(a @ dxx + b @ dxy)
298         + (1/beta)*(a @ dyy - b @ dxy))
299
300     v = (d @ 0q
301          + (1/alpha)*(a @ dxy + b @ dyy)
302          + (1/beta)*(-a @ dxy + b @ dxx))
303
304     return np.array([float(u), float(v)], dtype=float)
305
306     """
307     def vitesse(x1, y1, X, a, b, c, d, alpha=0.9, beta=0.1, epsilon=1e-9,
308 ↪      normalize=False):
309         """
310         x1, y1 : point de requête
311         X      : (N,2) points d'apprentissage (mêmes coords que celles passées à params)
312         a, b   : (N,) coefficients noyau
313         c, d   : (3,) coefficients affines (c[0] + c[1]*x + c[2]*y, idem pour d)
314         """
315         X = np.asarray(X, float)
316         Z = np.array([x1, y1], dtype=float) - X # (N,2)
317
318         # Évalue les dérivées du noyau : dxx, dyy, dxy (lambdify requis : fx, fy, fxy)
319         try:
320             dxx = fx(Z[:, 0], Z[:, 1], epsilon)
321             dyy = fy(Z[:, 0], Z[:, 1], epsilon)
322             dxy = fxy(Z[:, 0], Z[:, 1], epsilon)
323         except NameError:
324             # fallback si fx/fy/fxy n'existent pas : on garde subs (plus lent)
325             dxx = [float(phiix.subs({x: xv, y: yv})) for xv, yv in zip(Z[:,0], Z[:,1])]
326             dyy = [float(phiyy.subs({x: xv, y: yv})) for xv, yv in zip(Z[:,0], Z[:,1])]
327             dxy = [float(phixy.subs({x: xv, y: yv})) for xv, yv in zip(Z[:,0], Z[:,1])]
328
329         dxx = np.asarray(dxx, float)
330         dyy = np.asarray(dyy, float)
331         dxy = np.asarray(dxy, float)
332
333         # Nettoyage sécurité
334         dxx = np.nan_to_num(dxx, nan=0.0, posinf=0.0, neginf=0.0)
335         dyy = np.nan_to_num(dyy, nan=0.0, posinf=0.0, neginf=0.0)
336         dxy = np.nan_to_num(dxy, nan=0.0, posinf=0.0, neginf=0.0)
337
338         a = np.asarray(a, float).reshape(-1)

```

```

338     b = np.asarray(b, float).reshape(-1)
339
340     u = c[0] + x1 * c[1] + y1 * c[2]
341     v = d[0] + x1 * d[1] + y1 * d[2]
342
343     u += np.nansum(a * (1/alpha) * dxx + a * (1/beta) * dyy + b * ((1/alpha) -
    ↪ (1/beta)) * dxy)
344     v += np.nansum(a * ((1/alpha) - (1/beta)) * dxy + b * (1/beta) * dxx + b *
    ↪ (1/beta) * dyy)
345
346     return np.array([u, v], dtype=float)"""
347
348 def poly_deg2(X): # X: (N,2)
349     x = X[:,0]; y = X[:,1]
350     return np.column_stack([np.ones_like(x), x, y, x*x, x*y, y*y]) # (N,6)
351
352 def params_poly2(X, u, v, alpha=1.0, beta=1.0, epsilon=1e-6):
353     X = np.asarray(X, float); u = np.asarray(u, float); v = np.asarray(v, float)
354     N = X.shape[0]; assert X.shape[1]==2 and u.size==N and v.size==N
355
356     # matrices noyau vectorisées (tes fx/fy/fxy + diag=0 + sym)
357     U = X[:,0][:,None] - X[:,0][None,:]
358     V = X[:,1][:,None] - X[:,1][None,:]
359     dxx = np.nan_to_num(fx(U, V, epsilon), nan=0.0, posinf=0.0, neginf=0.0)
360     dyy = np.nan_to_num(fy(U, V, epsilon), nan=0.0, posinf=0.0, neginf=0.0)
361     dxy = np.nan_to_num(fxy(U, V, epsilon), nan=0.0, posinf=0.0, neginf=0.0)
362     np.fill_diagonal(dxx, 0.0); np.fill_diagonal(dyy, 0.0); np.fill_diagonal(dxy,
    ↪ 0.0)
363     Kd = np.block([[dxx, dxy],[dxy, dyy]]); Kd = 0.5*(Kd+Kd.T)
364     Kr = np.block([[dyy,-dxy],[-dxy, dxx]]); Kr = 0.5*(Kr+Kr.T)
365
366     O = poly_deg2(X) # (N,6)
367     P = np.kron(np.eye(2), O) # (2N, 12)
368     uv = np.concatenate([u, v]) # (2N,)
369
370     K = (1/alpha)*Kd + (1/beta)*Kr
371     A = np.block([[K, P],
372                   [P.T, np.zeros((12, 12))]])
373     B = np.concatenate([uv, np.zeros(12)])
374
375     try:
376         sol = np.linalg.solve(A, B)

```

```

377     except np.linalg.LinAlgError:
378         sol = np.linalg.lstsq(A, B, rcond=None)[0]
379
380     a = sol[0:N]
381     b = sol[N:2*N]
382     c = sol[2*N:2*N+6]
383     d = sol[2*N+6:2*N+12]
384     return a, b, c, d
385
386 def vitesse_poly2(x1, y1, X, a, b, c, d, alpha=1.0, beta=1.0, epsilon=1e-6,
↪ tol=1e-8):
387     X = np.asarray(X, float)
388     Zx = x1 - X[:, 0]
389     Zy = y1 - X[:, 1]
390     r2 = Zx*Zx + Zy*Zy
391     mask0 = r2 <= tol
392
393     with np.errstate(divide='ignore', invalid='ignore'):
394         dxx = fx(Zx, Zy, epsilon)
395         dyy = fy(Zx, Zy, epsilon)
396         dxy = fxy(Zx, Zy, epsilon)
397
398     # nettoyage + annulation des self-terms
399     dxx = np.nan_to_num(dxx); dyy = np.nan_to_num(dyy); dxy = np.nan_to_num(dxy)
400     dxx[mask0] = 0.0; dyy[mask0] = 0.0; dxy[mask0] = 0.0
401
402     Oq = np.array([1.0, x1, y1, x1*x1, x1*y1, y1*y1], float)
403
404     u = (c @ Oq
405          + (1/alpha)*(a @ dxx + b @ dxy)
406          + (1/beta)*(a @ dyy - b @ dxy))
407
408     v = (d @ Oq
409          + (1/alpha)*(a @ dxy + b @ dyy)
410          + (1/beta)*(-a @ dxy + b @ dxx))
411
412     return float(u), float(v)
413
414
415
416 lx = uo_0['longitude'].values if hasattr(uo_0, 'values') else uo_0['longitude']
417 ly = uo_0['latitude'].values if hasattr(uo_0, 'values') else uo_0['latitude']

```



```

418 lon_grid_new, lat_grid_new = np.meshgrid(lx, ly) # shape = (len(ly), len(lx))
419
420 def _build_training_set(u_da, v_da):
421     """Extrait X,u,v sur les points non-NaN de l'échantillon (même ordre)."""
422     # coords
423     lons = u_da['longitude'].values
424     lats = u_da['latitude'].values
425     LON, LAT = np.meshgrid(lons, lats) # (ny,nx)
426     U = np.asarray(u_da.values, float)
427     V = np.asarray(v_da.values, float)
428     mask = np.isfinite(U) & np.isfinite(V)
429     X = np.column_stack([LON[mask], LAT[mask]]).astype(float) # (N,2)
430     u = U[mask].astype(float).ravel(order='C')
431     v = V[mask].astype(float).ravel(order='C')
432     return X, u, v
433
434 def _reconstruct_field(lx, ly, X, a, b, c, d, alpha, beta, eps):
435     """Reconstruit (U,V) sur la grille (lx,ly) avec vitesse_poly2."""
436     lon_g, lat_g = np.meshgrid(lx, ly) # (ny,nx)
437     pts = np.column_stack([lon_g.ravel(order='C'), lat_g.ravel(order='C')])
438     Vpred = [vitesse_poly2(x, y, X, a, b, c, d, alpha=alpha, beta=beta, epsilon=eps,
439         ↪ tol=TOL)
440         for (x, y) in pts]
441     Vpred = np.array(Vpred, float)
442     U = Vpred[:,0].reshape(lat_g.shape) # shape (ny,nx)
443     V = Vpred[:,1].reshape(lat_g.shape)
444     return U, V
445
446 def _fit_and_reconstruct(u_da, v_da, lx, ly, alpha=ALPHA, beta=BETA, eps=EPS):
447     """Pipeline complet pour un échantillon : fit -> reconstruction."""
448     X, u, v = _build_training_set(u_da, v_da)
449     if X.shape[0] < 6:
450         raise ValueError("Échantillon trop petit pour l'ajustement (N < 6).")
451     a, b, c, d = params(X, u, v, alpha=alpha, beta=beta, epsilon=eps)
452     U_am, V_am = _reconstruct_field(lx, ly, X, a, b, c, d, alpha, beta, eps)
453     print(alpha)
454     return U_am, V_am
455
456     # Importation data
457     file = 'data/01-30.nc'
458     ds = xr.open_dataset('data/kg2.nc')
459     print(ds)

```

```

459
460 uo = uo_0.values # shape (temps, lat, lon)
461 vo = vo_0.values
462 lat = uo_0['latitude'].values
463 lon = uo_0['longitude'].values
464
465 skip = 4
466 lon_skip = lon[::skip]
467 lat_skip = lat[::skip]
468 lon_mesh, lat_mesh = np.meshgrid(lon_skip, lat_skip)
469
470 fig, ax = plt.subplots(figsize=(8, 6))
471 quiv = ax.quiver(lon_mesh, lat_mesh, uo[0, ::skip, ::skip], vo[0, ::skip, ::skip],
↳ angles='xy', color='blue')
472 ax.set_title("Champ de courants océaniques (animation)")
473 ax.set_xlim(lon.min(), lon.max())
474 ax.set_ylim(lat.min(), lat.max())
475
476 def update(frame):
477     quiv.set_UVC(uo[frame, ::skip, ::skip], vo[frame, ::skip, ::skip])
478     ax.set_title(f"Champ de courants océaniques - t =
↳ {str(uo_0['time'].values[frame])[10]}")
479     return quiv,
480
481 ani = FuncAnimation(fig, update, frames=uo.shape[0], interval=20000, blit=True)
482
483 # Sauvegarde
484 from matplotlib.animation import PillowWriter
485 ani.save("champ_vecteurs.gif", writer=PillowWriter(fps=3))
486
487
488 skip = 1
489 lon_uo, lat_uo = np.meshgrid(uo_0['longitude'][::skip], uo_0['latitude'][::skip])
490 lon_vo, lat_vo = np.meshgrid(vo_0['longitude'][::skip], vo_0['latitude'][::skip])
491
492 fig, ax = plt.subplots(1, 2, figsize=(15,6))
493 #uo_0.plot(ax=ax[0])
494 ax[0].quiver(lon_uo, lat_uo, uo_0[::skip, ::skip], vo_0[::skip, ::skip], color='b',
↳ angles='xy')
495 ax[0].set_xlim(ds['longitude'].values.min(), ds['longitude'].values.max())
496 ax[0].set_ylim(ds['latitude'].values.min(), ds['latitude'].values.max())
497 ax[0].set_title("Champ de courants océaniques")

```

```

498
499
500 skip = 4
501 lon_uo, lat_uo = np.meshgrid(uo_0['longitude'][:,::skip], uo_0['latitude'][:,::skip])
502 lon_vo, lat_vo = np.meshgrid(vo_0['longitude'][:,::skip], vo_0['latitude'][:,::skip])
503
504 #vo_0.plot(ax=ax[1])
505 ax[1].quiver(lon_vo, lat_vo, uo_0[:,::skip, ::skip], vo_0[:,::skip, ::skip], color='b',
506             ↪ angles='xy')
507 ax[1].set_xlim(ds['longitude'].values.min(), ds['longitude'].values.max())
508 ax[1].set_ylim(ds['latitude'].values.min(), ds['latitude'].values.max())
509
510 ax[1].set_title("Champ de courants océaniques sous échantillonné")
511
512 plt.show()
513
514 np.random.seed(42)
515 u25a, v25a = echantillon_aleatoire(uo_0, vo_0, 0.25)
516 u50a, v50a = echantillon_aleatoire(uo_0, vo_0, 0.50)
517 u33a, v33a = echantillon_aleatoire(uo_0, vo_0, 0.33)
518
519 u25s, v25s = echantillon_systematique(uo_0, vo_0, 0.25)
520 u50s, v50s = echantillon_systematique(uo_0, vo_0, 0.50)
521 u33s, v33s = echantillon_systematique(uo_0, vo_0, 0.33)
522
523 u25ab, v25ab = noise_von_mises(u25a, v25a, 5.0)
524 u50ab, v50ab = noise_von_mises(u50a, v50a, 5.0)
525 u33ab, v33ab = noise_von_mises(u33a, v33a, 5.0)
526
527 u25sb, v25sb = noise_von_mises(u25a, v25a, 5.0)
528 u50sb, v50sb = noise_von_mises(u50a, v50a, 5.0)
529 u33sb, v33sb = noise_von_mises(u33a, v33a, 5.0)
530
531 n = 100
532 s=1e-05
533 aae_list_a = []
534 eqm_list_a = []
535 aae_list_s = []
536 eqm_list_s = []
537 for kappa in np.linspace(5.0, 10.0, 100):
538     u25ab, v25ab = noise_von_mises(u25a, v25a, kappa=kappa)

```

```

539 u50ab, v50ab = noise_von_mises(u50a, v50a, kappa=kappa)
540 u33ab, v33ab = noise_von_mises(u33a, v33a, kappa=kappa)
541 u25ab_th, v25ab_th = interpolate_vector_field_from_xarray(u25ab, v25ab, lon_grid,
↳ lat_grid, method='thin_plate', s=s)
542 u50ab_th, v50ab_th = interpolate_vector_field_from_xarray(u50ab, v50ab, lon_grid,
↳ lat_grid, method='thin_plate', s=s)
543 u33ab_th, v33ab_th = interpolate_vector_field_from_xarray(u33ab, v33ab, lon_grid,
↳ lat_grid, method='thin_plate', s=s)
544
545
546
547 aae_list_a.append([float(np.round(aae(uo_0, vo_0, u25ab_th, v25ab_th),3)),
↳ float(np.round(aae(uo_0, vo_0, u33ab_th, v33ab_th),3)),
↳ float(np.round(aae(uo_0, vo_0, u50ab_th, v50ab_th),3))])
548 eqm_list_a.append([float(np.round(eqm(uo_0, vo_0, u25ab_th, v25ab_th),4)),
↳ float(np.round(eqm(uo_0, vo_0, u33ab_th, v33ab_th),4)),
↳ float(np.round(eqm(uo_0, vo_0, u50ab_th, v50ab_th),4))])
549
550 u25sb, v25sb = noise_von_mises(u25a, v25a, kappa=kappa)
551 u50sb, v50sb = noise_von_mises(u50a, v50a, kappa=kappa)
552 u33sb, v33sb = noise_von_mises(u33a, v33a, kappa=kappa)
553
554 u25sb_th, v25sb_th = interpolate_vector_field_from_xarray(u25sb, v25sb, lon_grid,
↳ lat_grid, method='thin_plate', s=s)
555 u50sb_th, v50sb_th = interpolate_vector_field_from_xarray(u50sb, v50sb, lon_grid,
↳ lat_grid, method='thin_plate', s=s)
556 u33sb_th, v33sb_th = interpolate_vector_field_from_xarray(u33sb, v33sb, lon_grid,
↳ lat_grid, method='thin_plate', s=s)
557
558 aae_list_s.append([float(np.round(aae(uo_0, vo_0, u25sb_th, v25sb_th),3)),
↳ float(np.round(aae(uo_0, vo_0, u33sb_th, v33sb_th),3)),
↳ float(np.round(aae(uo_0, vo_0, u50sb_th, v50sb_th),3))])
559 eqm_list_s.append([float(np.round(eqm(uo_0, vo_0, u25sb_th, v25sb_th),4)),
↳ float(np.round(eqm(uo_0, vo_0, u33sb_th, v33sb_th),4)),
↳ float(np.round(eqm(uo_0, vo_0, u50sb_th, v50sb_th),4))])
560
561 fig, axes = plt.subplots(2, 2, figsize=(15, 8))
562
563 axes = axes.flatten()
564
565 axes[0].boxplot(np.array(aae_list_a), labels=['25%', '33%', '50%'])

```

```

566 axes[0].set_ylabel('EAM (erreur angulaire moyenne en °)')
567 axes[0].set_title('Echantillonnage aléatoire - EAM')
568
569 axes[1].boxplot(np.array(eqm_list_a), labels=['25%', '33%', '50%'])
570 axes[1].set_ylabel('EQM (erreur quadratique moyenne en m²/s²)')
571 axes[1].set_title('Echantillonnage aléatoire - EQM')
572
573 axes[2].boxplot(np.array(aae_list_s), labels=['25%', '33%', '50%'])
574 axes[2].set_ylabel('EAM (erreur angulaire moyenne en °)')
575 axes[2].set_title('Echantillonnage systématique - EAM')
576
577 axes[3].boxplot(np.array(eqm_list_s), labels=['25%', '33%', '50%'])
578 axes[3].set_ylabel('EQM (erreur quadratique moyenne en m²/s²)')
579 axes[3].set_title('Echantillonnage systématique - EQM')
580
581 fig.suptitle('Analyse de la sensibilité au bruit', fontsize=16)
582 plt.tight_layout()
583 plt.show()
584
585 u25s_am, v25s_am = _fit_and_reconstruct(u25s, v25s, lx, ly)
586 u33s_am, v33s_am = _fit_and_reconstruct(u33s, v33s, lx, ly)
587 u50s_am, v50s_am = _fit_and_reconstruct(u50s, v50s, lx, ly)
588
589 u25a_am, v25a_am = _fit_and_reconstruct(u25a, v25a, lx, ly)
590 u33a_am, v33a_am = _fit_and_reconstruct(u33a, v33a, lx, ly)
591 u50a_am, v50a_am = _fit_and_reconstruct(u50a, v50a, lx, ly)
592
593 import numpy as np
594 import matplotlib.pyplot as plt
595
596 # listes pour stocker les erreurs
597 aae_A, eqm_A = [], []
598 aae_B, eqm_B = [], []
599
600 s = 1e-05
601 for kappa in np.linspace(5.0, 100.0, 100):
602     u_noisy, v_noisy = noise_von_mises(u50s, v50s, kappa=kappa)
603
604     # ===== Méthode A : thin-plate spline =====
605     uA, vA = interpolate_vector_field_from_xarray(
606         u_noisy, v_noisy, lon_grid, lat_grid, method='thin_plate', s=s
607     )

```

```

608     aae_A.append(aae(uo_0, vo_0, uA, vA))
609     eqm_A.append(eqm(uo_0, vo_0, uA, vA))
610
611     # ===== Méthode B : Amodei =====
612     uB, vB = _fit_and_reconstruct(u_noisy, v_noisy, lx, ly)
613     # vérifier que la forme correspond
614     if uB.shape != uo_0.shape:
615         uB, vB = uB.reshape(uo_0.shape), vB.reshape(vo_0.shape)
616     aae_B.append(aae(uo_0, vo_0, uB, vB))
617     eqm_B.append(eqm(uo_0, vo_0, uB, vB))
618
619     #---- Boxplots comparaison ----
620     plt.figure(figsize=(6,4))
621     plt.boxplot([aae_A, aae_B], labels=['Thin-Plate', 'Amodei'])
622     plt.ylabel("AAE")
623     plt.title("Sensibilité au bruit (AAE)")
624     plt.show()
625
626     plt.figure(figsize=(6,4))
627     plt.boxplot([eqm_A, eqm_B], labels=['Thin-Plate', 'Amodei'])
628     plt.ylabel("EQM")
629     plt.title("Sensibilité au bruit (EQM)")
630     plt.show()

```